Universidad del Quindío

Taller 3

Ordenamiento y Búsqueda

Comparación experimental de algoritmos de ordenamiento y búsqueda
en Java y Python

Santiago Orozco Zuluaga
Jhony Alexander Perea Perea

Curso: Análisis de Algoritmos
Docente: Carlos Andrés Flórez Villarraga

Repositorio:
`https://github.com/Shuh3n/taller3-ordenamiento-busqueda`

Octubre de 2026

Contenido

1. Presentación **2**

2. Metodología y condiciones del experimento **2**

3. Punto 1: algoritmos de ordenamiento **4**

3.1. Crecimiento del tiempo con n 5

3.2. Efecto del orden de entrada (lenguaje compilado: Java) 8

3.3. Compilado frente a interpretado 12

4. Punto 2: algoritmos de búsqueda **14**

4.1. Crecimiento del tiempo con n 14

4.2. Compilado frente a interpretado 18

4.3. Posición de la clave 18

5. Punto 3: análisis comparativo **21**

5.1. Comparativos frente a no comparativos 21

5.2. ¿Ordenar o no ordenar? 23

6. Comparación entre los dos equipos **25**

7. Uso de inteligencia artificial **25**

8. Conclusiones **26**

A. Tablas completas de resultados **28**

A.1. Equipo A 28

A.2. Equipo B 30

1. Presentación

Este informe documenta un estudio experimental de seis algoritmos de ordenamiento (Insertion Sort, Dual-Pivot QuickSort, Merge Sort, Heap Sort, Radix Sort y Bucket Sort) y seis algoritmos de búsqueda (lineal, binaria, ternaria, por saltos, por interpolación y exponencial). Cada algoritmo se implementó con la misma lógica en dos lenguajes: **Java** (compilado, con compilación JIT) y **Python** (interpretado), para un total de 24 implementaciones. El objetivo es medir su desempeño, verificar si el comportamiento observado corresponde con la complejidad teórica y explicar, a partir de los datos obtenidos, las diferencias entre algoritmos, entre tipos de entrada y entre lenguajes.

Cómo se hizo. Los datos de prueba se generaron una sola vez con una semilla fija y se guardaron en archivos de texto plano; todos los programas los leen desde esos archivos, de modo que cada algoritmo se evalúa sobre exactamente los mismos valores. Cada medición se repitió cinco veces y se reporta el promedio y la desviación estándar. Los resultados se almacenaron en archivos CSV y las gráficas y tablas de este documento se construyeron directamente a partir de ellos.

Dos equipos de cómputo. Todo el flujo experimental se ejecutó completo en **dos computadores distintos** (equipo A y equipo B), siguiendo el procedimiento de la sección 2. Cada conjunto de resultados sale de un único equipo, para que sus tiempos sean comparables entre sí. Las gráficas de cada punto se presentan para ambos equipos, y la sección 6 compara los dos conjuntos de resultados. Que dos máquinas independientes lleguen a las mismas conclusiones cualitativas es, además, una verificación de que los hallazgos no dependen de un equipo en particular.

Estructura del documento. La sección 2 describe la metodología y las condiciones del experimento; las secciones 3, 4 y 5 desarrollan los Puntos 1 (ordenamiento), 2 (búsqueda) y 3 (análisis comparativo) del taller; la sección 6 compara los dos equipos; después se declara el uso de inteligencia artificial y se presentan las conclusiones. Las tablas completas de resultados están en el Anexo A.

2. Metodología y condiciones del experimento

Implementación. Las implementaciones de Java y de Python siguen exactamente la misma lógica y se adaptaron de los artículos de GeeksforGeeks citados en la Tabla 1. Los ordenamientos trabajan *en sitio* sobre el arreglo recibido y las búsquedas devuelven un índice o -1. Dos adaptaciones merecen mencionarse:

- **Dual-Pivot QuickSort:** los pivotes son el primer y el último elemento del intervalo (si el primero es mayor, se intercambian), tal como pide el taller. No se usa ninguna estrategia de pivote aleatorio ni de mediana.
- **Bucket Sort:** se usan $k = n$ cubetas de igual ancho sobre el intervalo $[\text{mín}, \text{máx}]$ del arreglo. El valor v se asigna a la cubeta $\left\lfloor \dfrac{(v - \text{mín}) \cdot k}{\text{máx} - \text{mín} + 1} \right\rfloor$. Las cubetas se almacenan de forma contigua (conteo y suma de prefijos) en lugar de una lista por cubeta, y cada cubeta se ordena con Insertion Sort, como en GeeksforGeeks. Radix Sort es LSD en base 10, con un Counting Sort estable por dígito (8 pasadas para números de 8 dígitos).

Datos de prueba. Se generaron una sola vez con el programa `tools/generate_data.py` (semilla fija 42) y se guardaron en archivos de texto plano, un entero por línea. Para cada tamaño $n \in \{1.000; 10.000; 100.000; 1.000.000\}$ hay un arreglo de enteros aleatorios de 8 dígitos (entre

Tabla 1: Fuentes (GeeksforGeeks) utilizadas para cada algoritmo.

Algoritmo	Fuente
Insertion Sort	`https://www.geeksforgeeks.org/insertion-sort-algorithm/`
Dual-Pivot QuickSort	`https://www.geeksforgeeks.org/dual-pivot-quicksort/`
Merge Sort	`https://www.geeksforgeeks.org/merge-sort/`
Heap Sort	`https://www.geeksforgeeks.org/heap-sort/`
Radix Sort	`https://www.geeksforgeeks.org/radix-sort/`
Bucket Sort	`https://www.geeksforgeeks.org/bucket-sort-2/`
Búsqueda lineal	`https://www.geeksforgeeks.org/linear-search/`
Búsqueda binaria	`https://www.geeksforgeeks.org/binary-search/`
Búsqueda ternaria	`https://www.geeksforgeeks.org/ternary-search/`
Búsqueda por saltos	`https://www.geeksforgeeks.org/jump-search/`
Búsqueda por interpolación	`https://www.geeksforgeeks.org/interpolation-search/`
Búsqueda exponencial	`https://www.geeksforgeeks.org/exponential-search/`

10.000.000 y 99.999.999), sus versiones ordenadas ascendente y descendentemente, y un archivo de 1.000 claves de búsqueda (500 presentes en el arreglo y 500 ausentes, mezcladas). Los programas leen esos archivos antes de medir; la lectura y escritura de archivos nunca entra en el tiempo medido.

Medición del tiempo. Se mide únicamente la ejecución del algoritmo con `System.nanoTime()` en Java y `time.perf_counter_ns()` en Python. Cada medición se repite 5 veces, *cada una sobre una copia nueva del arreglo original* (la copia no se cronometra, de modo que la segunda repetición no ordena un arreglo ya ordenado), y se reportan el promedio y la desviación estándar muestral. En Java se ejecuta antes un calentamiento (10 ejecuciones sobre 10.000 elementos por algoritmo de ordenamiento y 300 pasadas del conjunto de claves por algoritmo de búsqueda) que no entra en el promedio. Después de cada ejecución, fuera del tiempo medido, se verifica que el arreglo quedó ordenado (y con la misma suma de elementos) o que cada búsqueda devolvió el resultado correcto.

Mediciones de búsqueda. Una sola búsqueda dura tan poco que medirla de forma aislada estaría dominado por el ruido del reloj. Por eso se busca el conjunto de 1.000 claves en una sola pasada cronometrada y se reporta el *tiempo promedio por búsqueda*. Para el efecto de la posición de la clave, se busca 1.000 veces una misma clave ubicada en el índice 0 (inicio), $n/2$ (centro) o $n-1$ (final) del arreglo de $n = 1.000.000$.

Valores estimados y fallos. Antes de ejecutar un tamaño n, el programa estima su tiempo escalando el último promedio medido con el modelo de costo teórico del algoritmo:

$$t(n) \;=\; t(n_{\text{prev}}) \cdot \frac{f(n)}{f(n_{\text{prev}})}, \qquad f(n) \in \{n,\; n\log_2 n,\; n^2\}. \tag{1}$$

Si la estimación supera 300 s por ejecución, el algoritmo no se ejecuta y el valor se registra como *estimado*. Si un algoritmo falla (desbordamiento de pila en Java, `RecursionError` en Python), el error se registra sin ocultarlo y los tamaños mayores se estiman con el mismo escalamiento. En las gráficas de líneas los valores estimados se dibujan con línea punteada y marcador hueco; en las de barras, con barra rayada y el prefijo «∼»; en las tablas, con «∼» y «(est.)». Todas las gráficas usan escala logarítmica en el eje del tiempo. El registro completo de estimados y fallos de cada equipo está en las Tablas 18 y 23 del anexo.

Reproducción. El código fuente, los resultados y el programa que genera los datos con semilla fija están en `https://github.com/Shuh3n/taller3-ordenamiento-busqueda`. Las tablas de este documento se generan con `tools/make_latex.py` a partir de `results/` (equipo A) y `resultsv2/` (equipo B).

Formato de resultados. Cada fila de los CSV contiene: lenguaje, algoritmo, variante de entrada (o escenario de búsqueda), n, el tiempo de cada una de las cinco repeticiones, promedio, desviación estándar, si el valor fue medido o estimado, y el estado de la ejecución.

Entorno de pruebas. La Tabla 2 resume los dos equipos. Las corridas se hicieron con los benchmarks ejecutados uno tras otro (nunca en paralelo) Ambos equipos estuvieron conectados a la corriente. El equipo B no tenía otros programas en uso durante la corrida; en el equipo A había algunas aplicaciones abiertas, lo que pudo introducir algo de ruido en sus tiempos.

Tabla 2: Entorno de pruebas de cada equipo.

	Equipo A (`results/`)	Equipo B (`resultsv2/`)
Equipo	MacBook Air (M4)	ASUS Vivobook X1605ZA
Sistema operativo	macOS 26.6.2	Fedora Linux 44 (Workstation)
Procesador	Apple M4 (arm64)	Intel Core i5-12500H, 12.ª gen., 16 hilos
Memoria RAM	16 GB	16 GB
Almacenamiento	SSD de 256 GB	SSD M.2 de 1 TB
Java (JIT)	27.0.1.0.2	OpenJDK 25.0.4.1 (Red Hat)
Python	3.15.1	3.14.8

3. Punto 1: algoritmos de ordenamiento

La Tabla 3 resume la complejidad teórica investigada para cada algoritmo, su estabilidad y si trabaja *in-place*.

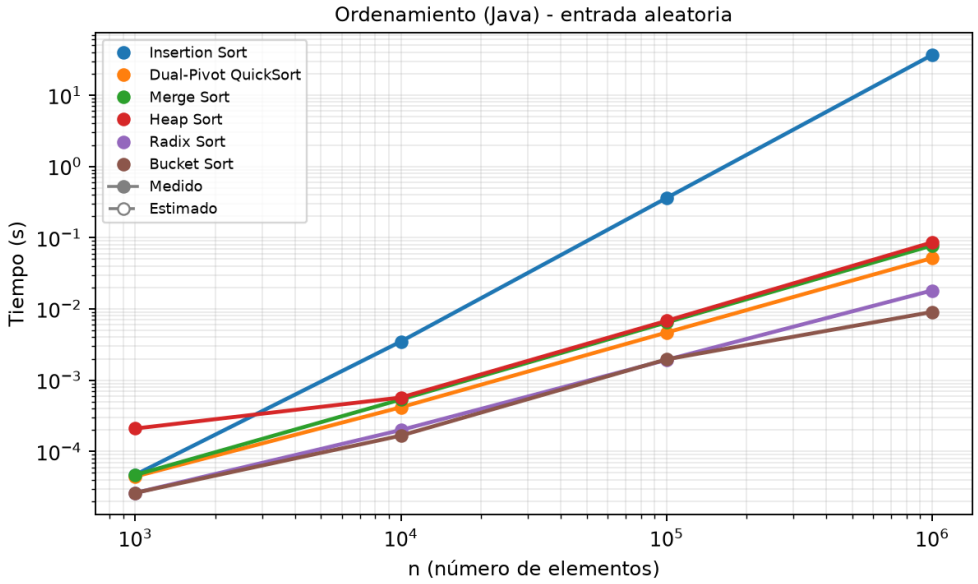
Tabla 3: Complejidad teórica, estabilidad y uso de memoria (n elementos, $d = 8$ dígitos, $b = 10$ base, k cubetas). *Estable*: conserva el orden relativo de claves iguales. *In-place*: usa memoria auxiliar $O(1)$ (o $O(\log n)$ de pila).

Algoritmo	Mejor caso	Caso promedio	Peor caso	Estable	In-place
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sí	Sí
Dual-Pivot QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No	Sí (pila $O(\log n)$ prom.)
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Sí	No ($O(n)$ auxiliar)
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No	Sí
Radix Sort (LSD, base 10)	$O(d(n+b))$	$O(d(n+b))$	$O(d(n+b))$	Sí	No ($O(n+b)$ auxiliar)
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$	Sí (con inserción estable)	No ($O(n+k)$ auxiliar)

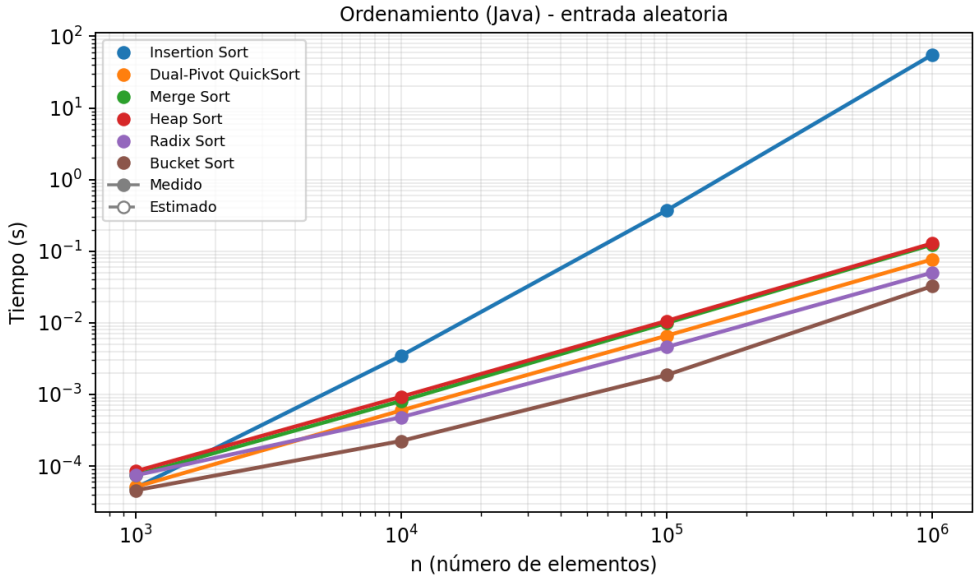
Dos observaciones sobre esta tabla, relevantes para lo que sigue. Primero, con pivotes fijos en los extremos, el peor caso de Dual-Pivot QuickSort aparece justamente con una entrada ya ordenada: el pivote izquierdo es el mínimo y el derecho el máximo del intervalo, por lo que cada partición solo descarta dos elementos y la recursión llega a una profundidad cercana a $n/2$. Segundo, el peor caso de Bucket Sort ocurre cuando casi todos los elementos caen en una misma cubeta, pues entonces el Insertion Sort interno trabaja sobre casi todo el arreglo.

3.1. Crecimiento del tiempo con n

Las Figuras 1 y 2 muestran el tiempo contra n con entrada aleatoria, una línea por algoritmo y escala logarítmica en ambos ejes. En una gráfica log-log, un algoritmo $O(n^2)$ se ve como una recta de pendiente ≈ 2 y uno $O(n)$ u $O(n \log n)$ como una recta de pendiente ≈ 1.



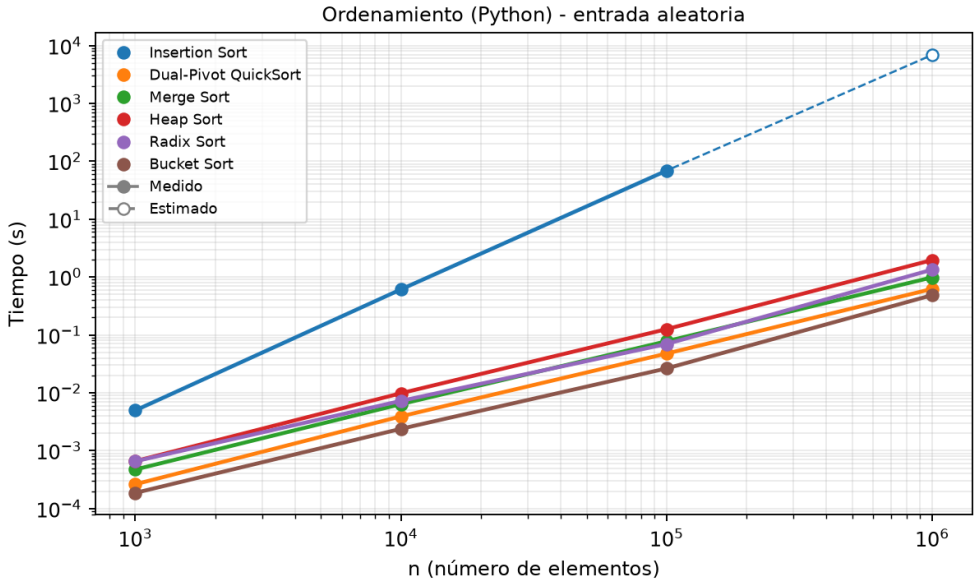
(a) Equipo A.



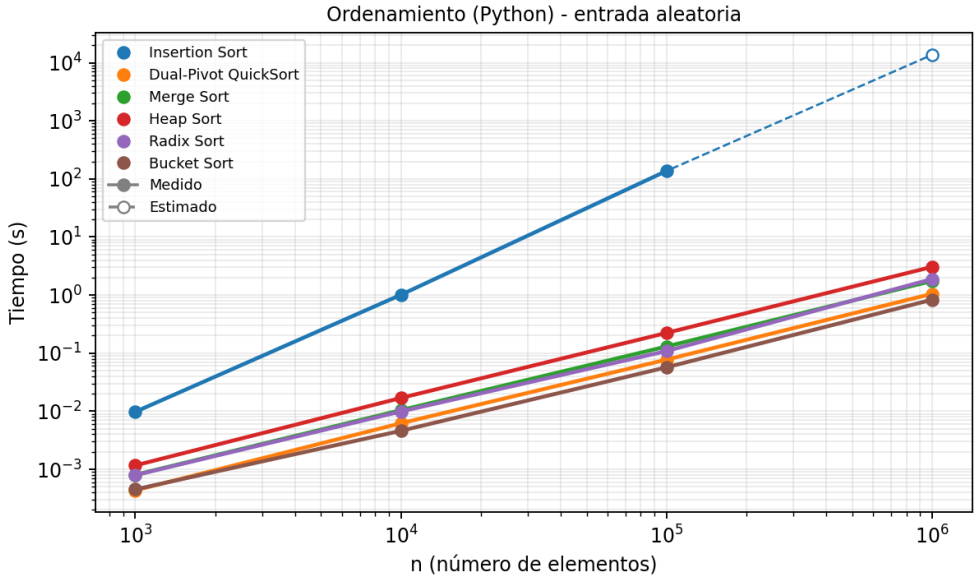
(b) Equipo B.

Figura 1: Ordenamiento en Java, entrada aleatoria (log-log). Medido: línea continua; estimado: línea punteada y marcador hueco.

La Tabla 4 muestra, para cada algoritmo, *cuántas veces aumentó el tiempo medio al multiplicar n por 10*, junto al factor que predice la complejidad teórica: $\times 100$ para $O(n^2)$, $\times 10$ para $O(n)$ y $\times 13{,}3$, $\times 12{,}5$ y $\times 12$ para $O(n \log n)$ en cada uno de los tres saltos.



(a) Equipo A.



(b) Equipo B.

Figura 2: Ordenamiento en Python, entrada aleatoria (log-log). En Python, Insertion Sort con $n = 1.000.000$ es un valor estimado.

Tabla 4: Factor de crecimiento del tiempo medio al multiplicar n por 10 (entrada aleatoria) frente al factor que predice la complejidad teórica. «∼» indica que alguno de los extremos es estimado.

Equipo A

Java

Algoritmo	1.000 → 10.000	10.000 → 100.000	100.000 → 1.000.000
Insertion Sort	×74,2 (teór. ×100)	×103 (teór. ×100)	×102 (teór. ×100)
Dual-Pivot QuickSort	×9,36 (teór. ×13,3)	×11,2 (teór. ×12,5)	×11,1 (teór. ×12)
Merge Sort	×11,5 (teór. ×13,3)	×12 (teór. ×12,5)	×12 (teór. ×12)
Heap Sort	×2,72 (teór. ×13,3)	×11,9 (teór. ×12,5)	×12,6 (teór. ×12)
Radix Sort	×7,63 (teór. ×10)	×9,64 (teór. ×10)	×9,42 (teór. ×10)
Bucket Sort	×6,36 (teór. ×10)	×11,7 (teór. ×10)	×4,63 (teór. ×10)

Python

Algoritmo	1.000 → 10.000	10.000 → 100.000	100.000 → 1.000.000
Insertion Sort	×123 (teór. ×100)	×113 (teór. ×100)	∼×100 (teór. ×100)
Dual-Pivot QuickSort	×14,9 (teór. ×13,3)	×12,2 (teór. ×12,5)	×13,1 (teór. ×12)
Merge Sort	×13,5 (teór. ×13,3)	×12,2 (teór. ×12,5)	×12,7 (teór. ×12)
Heap Sort	×14,9 (teór. ×13,3)	×12,8 (teór. ×12,5)	×15,6 (teór. ×12)
Radix Sort	×11 (teór. ×10)	×9,63 (teór. ×10)	×19,3 (teór. ×10)
Bucket Sort	×12,8 (teór. ×10)	×11,1 (teór. ×10)	×18,5 (teór. ×10)

Equipo B

Java

Algoritmo	1.000 → 10.000	10.000 → 100.000	100.000 → 1.000.000
Insertion Sort	×70,2 (teór. ×100)	×107 (teór. ×100)	×149 (teór. ×100)
Dual-Pivot QuickSort	×11,7 (teór. ×13,3)	×11,1 (teór. ×12,5)	×11,6 (teór. ×12)
Merge Sort	×10,8 (teór. ×13,3)	×12,3 (teór. ×12,5)	×12,4 (teór. ×12)
Heap Sort	×11 (teór. ×13,3)	×11,5 (teór. ×12,5)	×12,1 (teór. ×12)
Radix Sort	×6,49 (teór. ×10)	×9,55 (teór. ×10)	×10,9 (teór. ×10)
Bucket Sort	×4,89 (teór. ×10)	×8,34 (teór. ×10)	×17,4 (teór. ×10)

Python

Algoritmo	1.000 → 10.000	10.000 → 100.000	100.000 → 1.000.000
Insertion Sort	×104 (teór. ×100)	×136 (teór. ×100)	∼×100 (teór. ×100)
Dual-Pivot QuickSort	×14,3 (teór. ×13,3)	×12,5 (teór. ×12,5)	×13,6 (teór. ×12)
Merge Sort	×13 (teór. ×13,3)	×12,5 (teór. ×12,5)	×13,5 (teór. ×12)
Heap Sort	×14,5 (teór. ×13,3)	×13,2 (teór. ×12,5)	×13,7 (teór. ×12)
Radix Sort	×12,5 (teór. ×10)	×11 (teór. ×10)	×17,4 (teór. ×10)
Bucket Sort	×10,2 (teór. ×10)	×12,4 (teór. ×10)	×14,6 (teór. ×10)

Análisis.

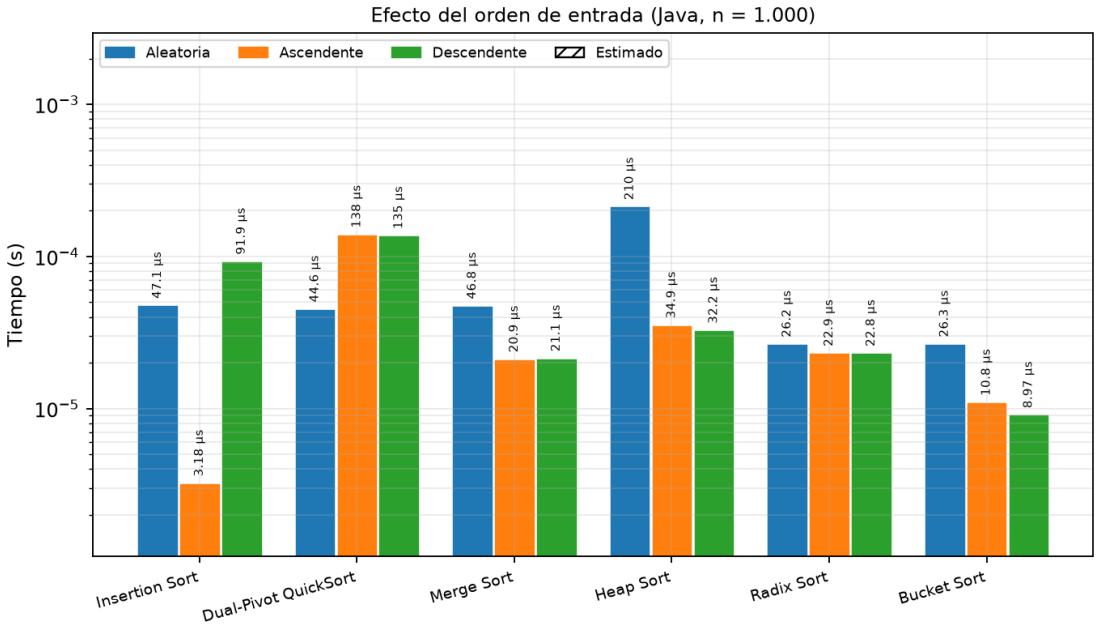
- **Insertion Sort** es el único algoritmo cuadrático y se nota: en Java, el tiempo crece ×102 y ×103 en los dos últimos saltos del equipo A (teórico ×100), y ×107 y ×149 en el equipo B. Con $n = 1.000.000$ tarda 36,7 s en A y 55,3 s en B, frente a 51,7 ms y 76,9 ms de Dual-Pivot QuickSort: ordena el mismo arreglo 700 veces más lento. El ×149 del equipo B (mayor que 100) y los ×113 y ×136 que se miden en Python entre 10^4 y 10^5 sugieren que, al crecer el arreglo, aparece un costo adicional por jerarquía de memoria (el arreglo de 10^6 enteros deja de caber en las cachés más rápidas); es una explicación plausible, no la verificamos con un perfilador.
- **Merge Sort, Heap Sort y Dual-Pivot QuickSort** (todos $O(n \log n)$ sobre datos aleatorios) crecen ×11 a ×13 por cada salto de 10 en Java a partir de $n = 10.000$, muy cerca del factor teórico de ×12 a ×12,5. En Python los factores son algo mayores (×12 a ×15,6). Que el factor medido esté ligeramente por encima de ×12 es esperable: la complejidad teórica cuenta operaciones, no accesos a memoria, y con arreglos grandes los accesos pierden localidad.
- **Radix Sort y Bucket Sort** deberían crecer ×10 ($O(d(n + b))$ con $d = 8$ fijo y $O(n + k)$ con $k = n$). En Java el factor es ×9,4 a ×9,6 para Radix Sort en el equipo A, y ×10,9 en B. Bucket Sort es el más irregular (×4,6 a ×17,4): sus accesos a memoria son dispersos (cada elemento se escribe en la posición de su cubeta), por lo que su costo por elemento depende mucho de si el arreglo cabe en caché. En Python ambos superan ×10 en el último salto (×14,6 a ×19,3), con el mismo tipo de explicación: una lista de 10^6 enteros de Python son 10^6 objetos dispersos en memoria, y las escrituras dispersas fallan más en caché.
- **Efecto de n pequeño.** Los saltos $1.000 \to 10.000$ de Java se desvían más: Heap Sort muestra ×2,7 en el equipo A cuando la teoría pide ×13,3. La causa más probable es que la medición de $n = 1.000$ (unas decenas de microsegundos) aún incluye costos del JIT que el calentamiento de 10 ejecuciones no alcanzó a absorber: ese primer valor es demasiado alto y el factor sale artificialmente bajo.

Cómo se estimaron los casos que no terminaron. Se usó la Ecuación (1). Hubo dos grupos (listados en las Tablas 18 y 23):

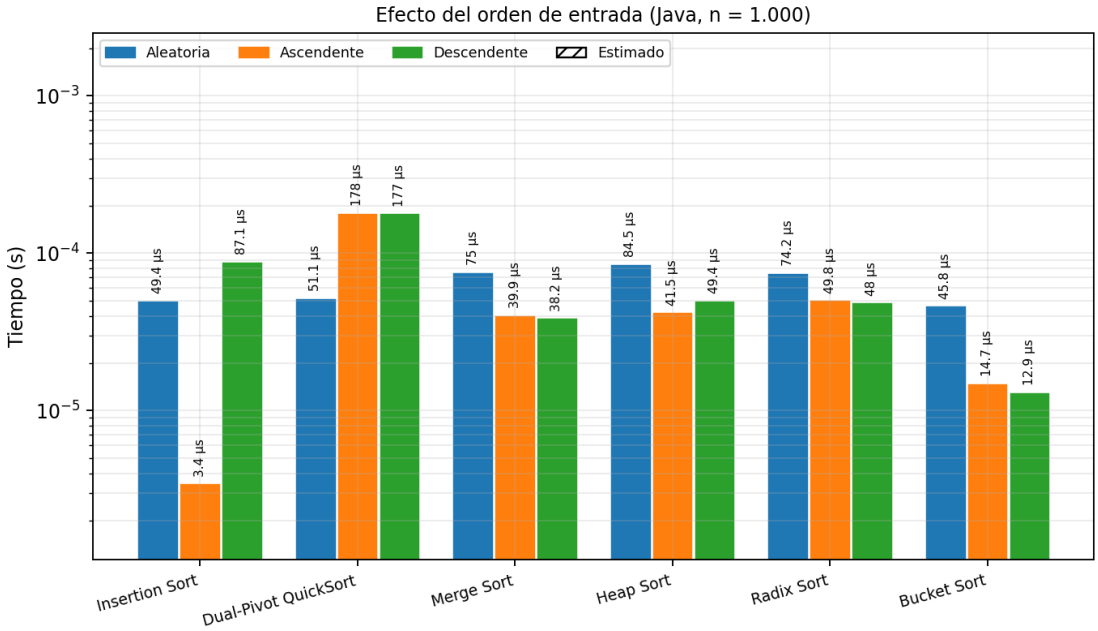
1. *Python, Insertion Sort con $n = 1.000.000$* (entrada aleatoria y descendente): se escaló con $f(n) = n^2$ desde $n = 100.000$ (×100). Resultan ∼6.890 s (≈ 1,9 h) y ∼12.400 s (≈ 3,4 h) en el equipo A, y ∼13.700 s (≈ 3,8 h) y ∼22.100 s (≈ 6,1 h) en el B; todos superan el límite de 5 minutos. Como en las medidas reales de Python el salto $10^4 \to 10^5$ fue ×113 (A) y ×136 (B), y no exactamente ×100, estas estimaciones deben leerse como una *cota inferior*.
2. *Dual-Pivot QuickSort con entrada ascendente o descendente*: el algoritmo falló (Java: `StackOverflowError` con $n = 100.000$; Python: `RecursionError` con $n = 10.000$, límite de recursión 1.000), por lo que los tamaños mayores se estimaron con $f(n) = n^2$ desde el último tamaño que sí terminó. La validez de ese modelo se puede comprobar en Java, donde $n = 1.000 \to 10.000$ sí se midió: el tiempo creció ×106 en A y ×92 en B, es decir, cuadráticamente.

3.2. Efecto del orden de entrada (lenguaje compilado: Java)

Las Figuras 3, 4 y 5 comparan, para $n = 1.000$, 10.000 y 100.000, los tres tipos de entrada (aleatoria, ascendente y descendente) en Java. Las barras rayadas son valores estimados; sobre cada barra se muestra su tiempo.

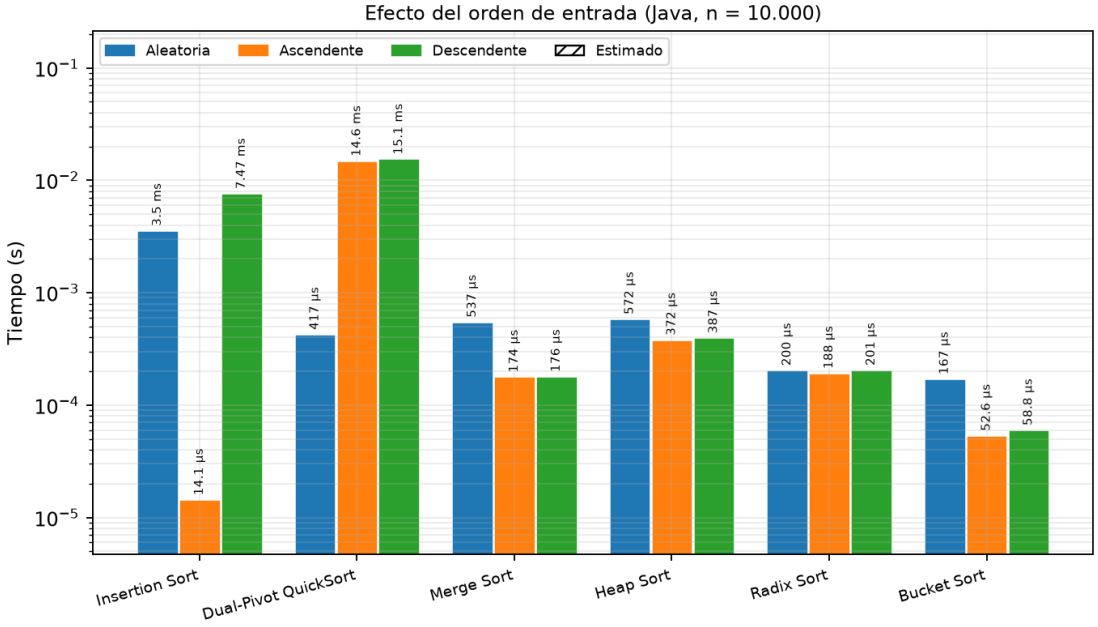


(a) Equipo A.

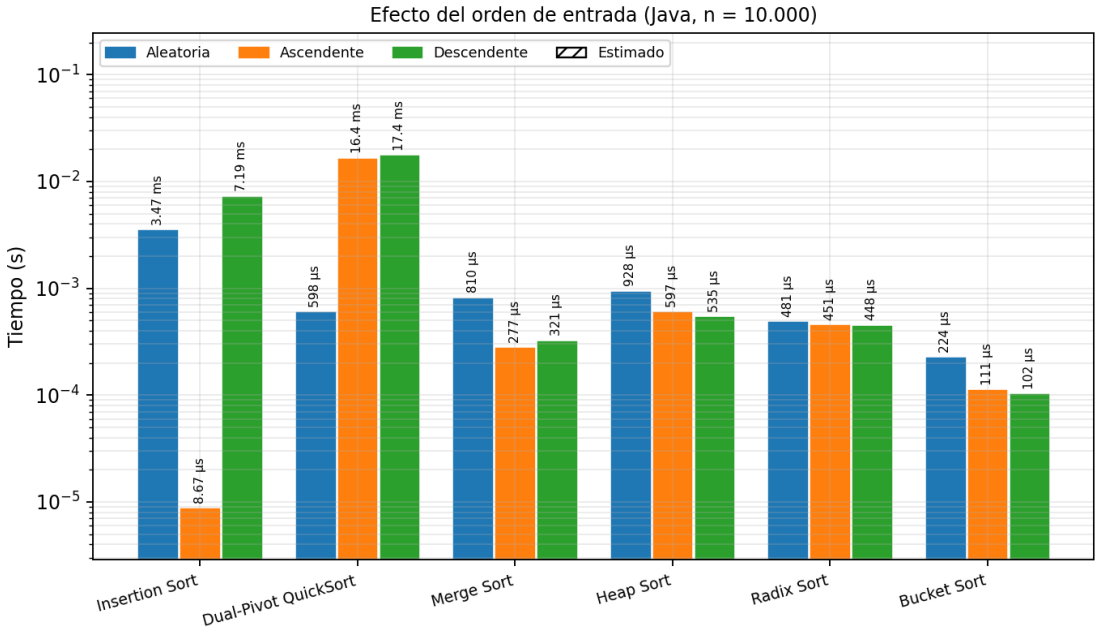


(b) Equipo B.

Figura 3: Efecto del orden de entrada en Java, $n = 1.000$.

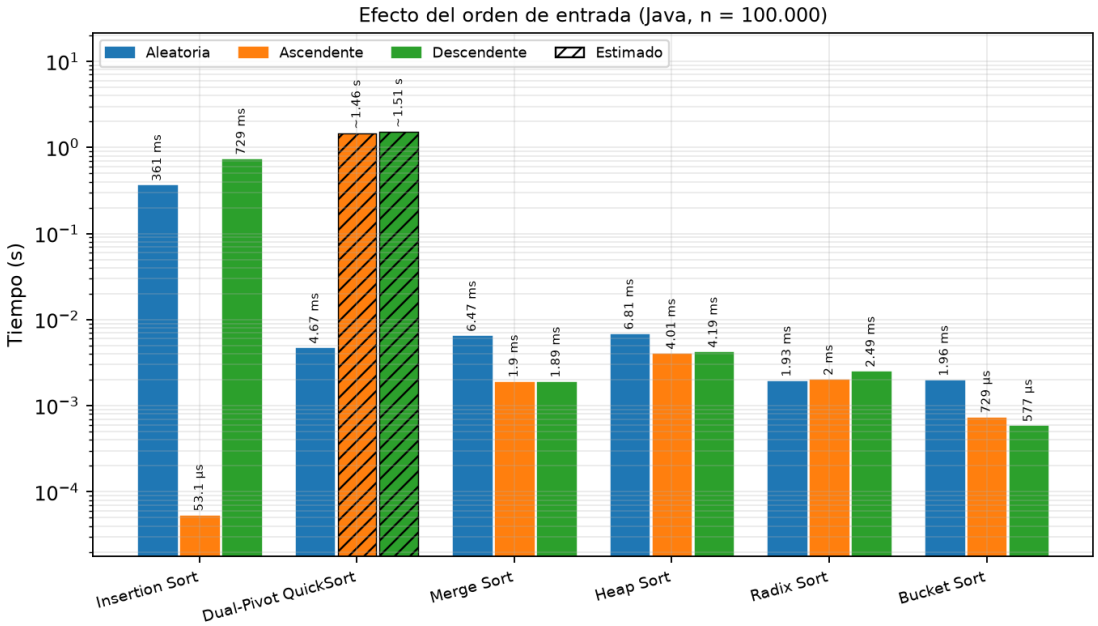


(a) Equipo A.

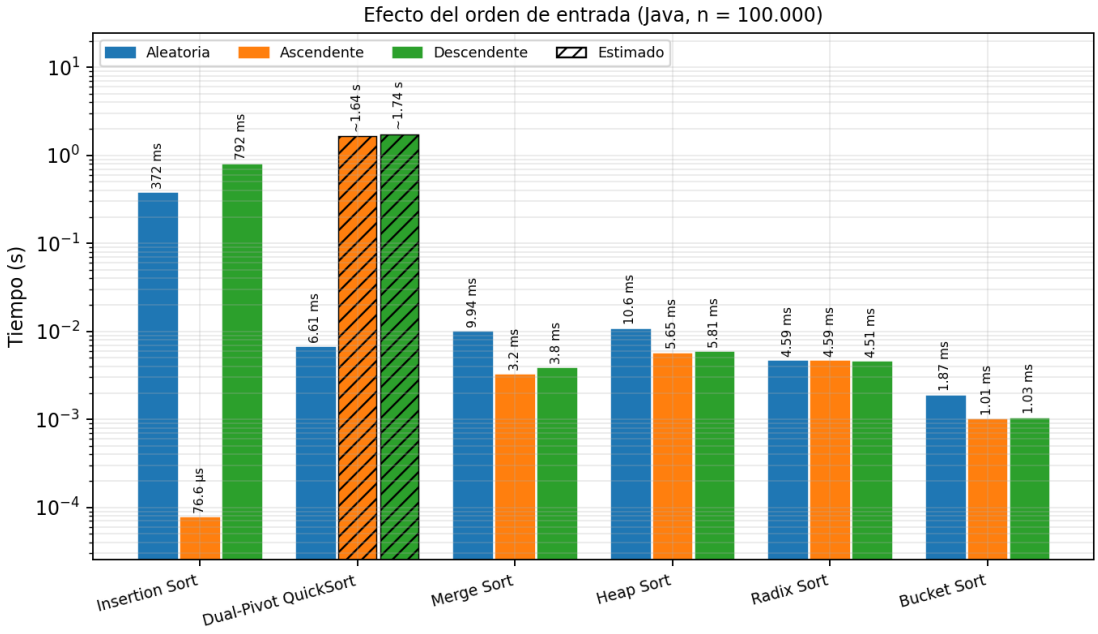


(b) Equipo B.

Figura 4: Efecto del orden de entrada en Java, $n = 10.000$.



(a) Equipo A.



(b) Equipo B.

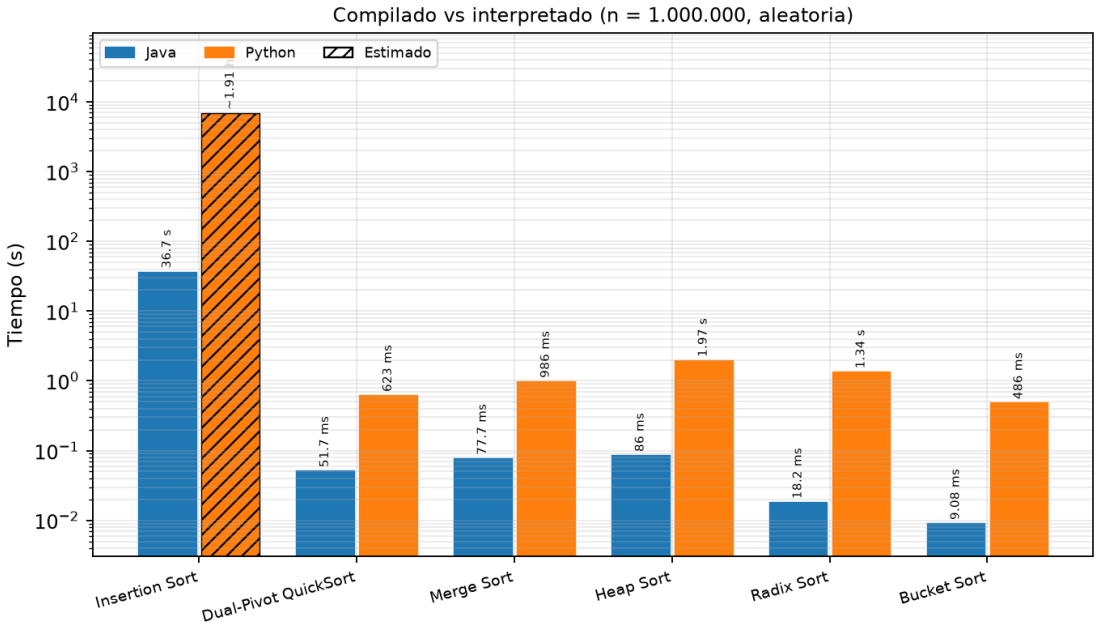
Figura 5: Efecto del orden de entrada en Java, $n = 100.000$. Las barras rayadas de Dual-Pivot QuickSort son estimadas: el algoritmo falló con `StackOverflowError`.

Análisis.

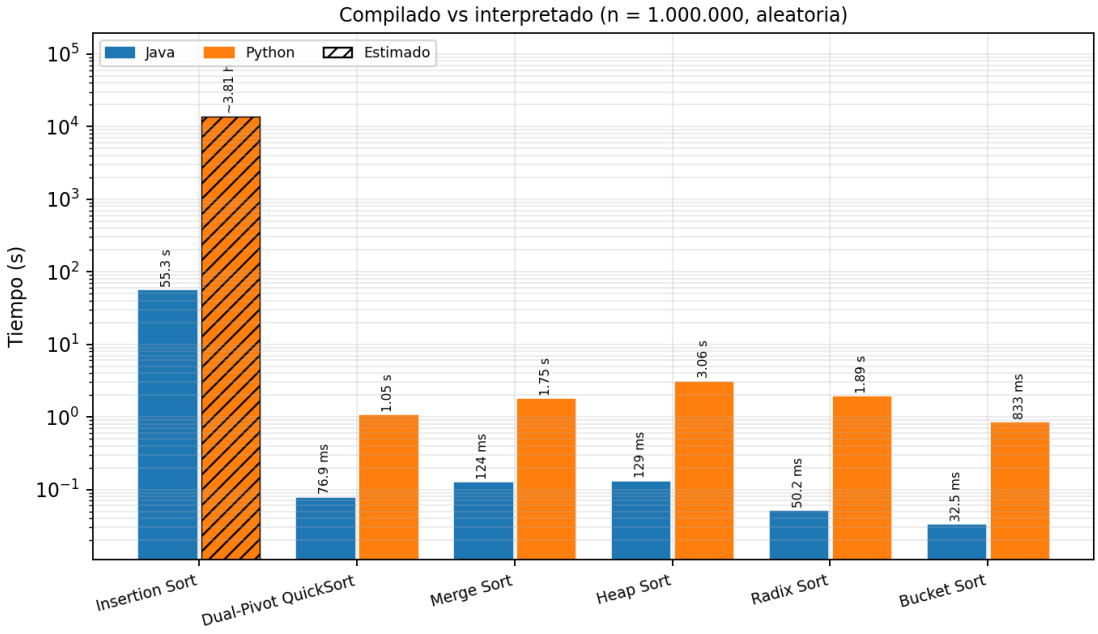
- **Insertion Sort cambió radicalmente.** Con entrada ascendente es su mejor caso, $O(n)$: cada elemento ya está en su lugar y el ciclo interno no desplaza nada. Con $n = 100.000$ tardó 53,1 μs frente a 361 ms con entrada aleatoria en el equipo A (casi 6.800 veces menos), y 76,6 μs frente a 372 ms en el B. Con entrada descendente es su peor caso, $O(n^2)$ completo: cada elemento debe recorrer todos los anteriores y tarda $\approx$ 1,8 a 2,1 veces más que con datos aleatorios (en promedio, con datos aleatorios cada elemento solo se desplaza la mitad del camino).
- **Dual-Pivot QuickSort fue el más afectado, y en sentido contrario.** Con la entrada ya ordenada (en cualquier dirección) los pivotes extremos son el mínimo y el máximo, la partición queda totalmente desbalanceada y el algoritmo degenera a $O(n^2)$: con $n = 1.000$ ya es unas 3 veces más lento que con datos aleatorios, con $n = 10.000$ es $\times$35 (A) y $\times$27 (B), y con $n = 100.000$ la profundidad de recursión ($\approx n/2 = 50.000$ llamadas) agotó la pila de Java con un `StackOverflowError`. Los tiempos de $n = 100.000$ de las barras rayadas son estimados con $f(n) = n^2$ ($\sim$1,46 s en A y $\sim$1,64 s en B frente a 4,67 ms y 6,61 ms con entrada aleatoria, unas 250 a 310 veces más). Es la razón por la que las implementaciones reales eligen el pivote al azar, por mediana de tres o por muestreo.
- **Merge Sort, Heap Sort y Bucket Sort mejoraron moderadamente con entrada ordenada** (aproximadamente entre 0,3 y 0,65 veces el tiempo de la entrada aleatoria; la excepción es Heap Sort con $n = 1.000$ en A, $\approx$ 0,16, afectado por el calentamiento). Su complejidad no cambia —Merge Sort y Heap Sort siempre hacen $\Theta(n \log n)$ operaciones—, pero con datos ordenados las comparaciones son muy predecibles (en la mezcla siempre sale primero un mismo lado, el procesador acierta las ramificaciones) y los accesos a memoria son más secuenciales. En Bucket Sort, un arreglo ordenado asigna los elementos a cubetas consecutivas, lo que vuelve secuencial la escritura que con datos aleatorios es dispersa. Son efectos de constantes, no de complejidad.
- **Radix Sort fue el único insensible al orden** (cociente entre 0,65 y 1,29 en todos los tamaños): siempre hace las mismas 8 pasadas de Counting Sort sobre los dígitos, y el costo de cada pasada no depende de cómo estén ordenados los valores, porque nunca compara dos elementos entre sí.
- **Peor comportamiento.** Entre los algoritmos que terminaron, el peor tiempo absoluto fue Insertion Sort con entrada descendente ($n = 1.000.000$: 73,8 s en A y 93,2 s en B). Entre los que *no* terminaron, el peor comportamiento fue Dual-Pivot QuickSort con entrada ya ordenada: lo provocó la combinación de pivotes en los extremos (partición de tamaño $n - 2$, recursión de profundidad $\approx n/2$). El mismo patrón aparece en Python, pero antes: el límite de recursión de 1.000 hace fallar el algoritmo con $n = 10.000$.

3.3. Compilado frente a interpretado

La Figura 6 compara Java y Python con $n = 1.000.000$ y entrada aleatoria, con dos barras por algoritmo (una por lenguaje).



(a) Equipo A.



(b) Equipo B.

Figura 6: Java frente a Python, $n = 1.000.000$, entrada aleatoria. La barra de Python de Insertion Sort es un valor estimado.

Análisis. Java fue más rápido en todos los algoritmos, pero por factores muy distintos (cociente Python/Java en el equipo A, y entre paréntesis en el B): Dual-Pivot QuickSort ×12 (×14), Merge Sort ×13 (×14), Heap Sort ×23 (×24), Bucket Sort ×54 (×26), Radix Sort ×74 (×38) e Insertion Sort ∼ ×188 (∼ ×248, con la barra de Python estimada).

- **Compilación JIT frente a interpretación.** El JIT de Java traduce los métodos calientes a código de máquina y los optimiza; Python ejecuta cada operación como instrucciones de bytecode que el intérprete despacha una a una, con comprobación de tipos en cada operación.
- **Representación de los datos.** En Java, `int[]` es un bloque contiguo de enteros de 4 bytes. En Python, una lista guarda *referencias* a objetos `int` dispersos por el montón: leer un elemento es seguir un puntero, lo que cuesta más y aprovecha peor la caché.
- **Por qué la brecha es mayor en unos algoritmos.** Es mayor donde el trabajo es una cantidad enorme de iteraciones simples sin ramificaciones impredecibles: Insertion Sort (ciclo interno con un desplazamiento por iteración), Radix Sort (8 pasadas × 3 ciclos sobre 10^6 elementos) y Bucket Sort. En Java esos ciclos corren casi a la velocidad de la memoria; en Python cada iteración cuesta decenas de nanosegundos de despacho de bytecode. Es menor en Dual-Pivot QuickSort y Merge Sort: ahí parte del tiempo en Java se va en costos que no desaparecen en Python (fallos de predicción de ramificaciones en las comparaciones, recursión) y en Python algunas operaciones se resuelven en C (en la mezcla, las copias con *slicing*), de manera que la distancia relativa se reduce. Esta es nuestra interpretación de los datos; no la medimos de forma aislada.

4. Punto 2: algoritmos de búsqueda

Tabla 5: Complejidad teórica de los algoritmos de búsqueda sobre un arreglo ordenado de n elementos.

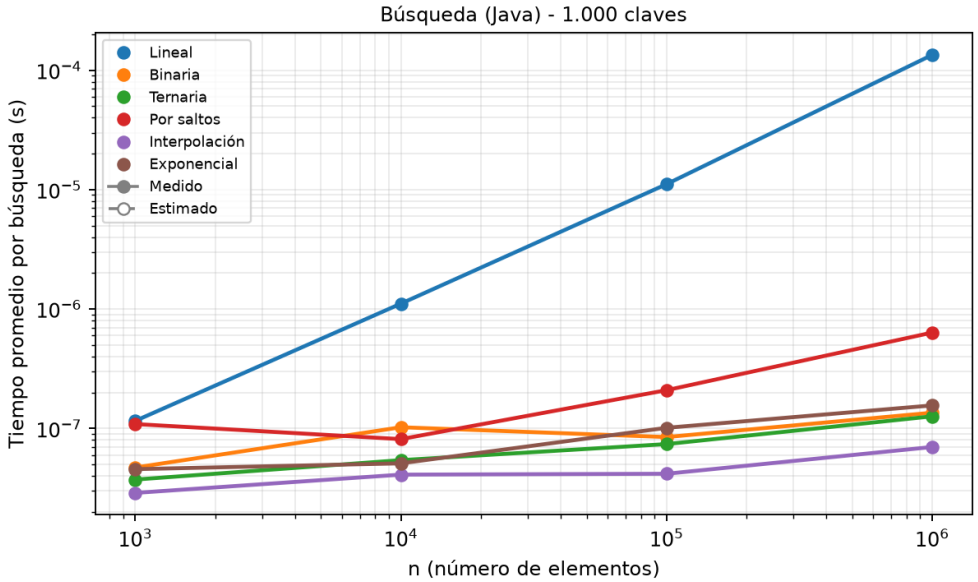
Algoritmo	Mejor caso	Peor caso / promedio	Supuesto
Lineal	$O(1)$	$O(n)$	Ninguno (no requiere orden)
Binaria	$O(1)$	$O(\log_2 n)$	Arreglo ordenado
Ternaria	$O(1)$	$O(\log_3 n)$ iteraciones (2 comparaciones c/u)	Arreglo ordenado
Por saltos	$O(1)$	$O(\sqrt{n})$	Arreglo ordenado
Interpolación	$O(1)$	promedio $O(\log \log n)$; peor caso $O(n)$	Ordenado y distribución uniforme
Exponencial	$O(1)$	$O(\log i)$, i = posición de la clave	Arreglo ordenado

Los arreglos de entrada son los ordenados ascendentemente del Punto 1. Cada medición busca el conjunto de 1.000 claves (500 presentes y 500 ausentes), cronometra el total y reporta el promedio por búsqueda.

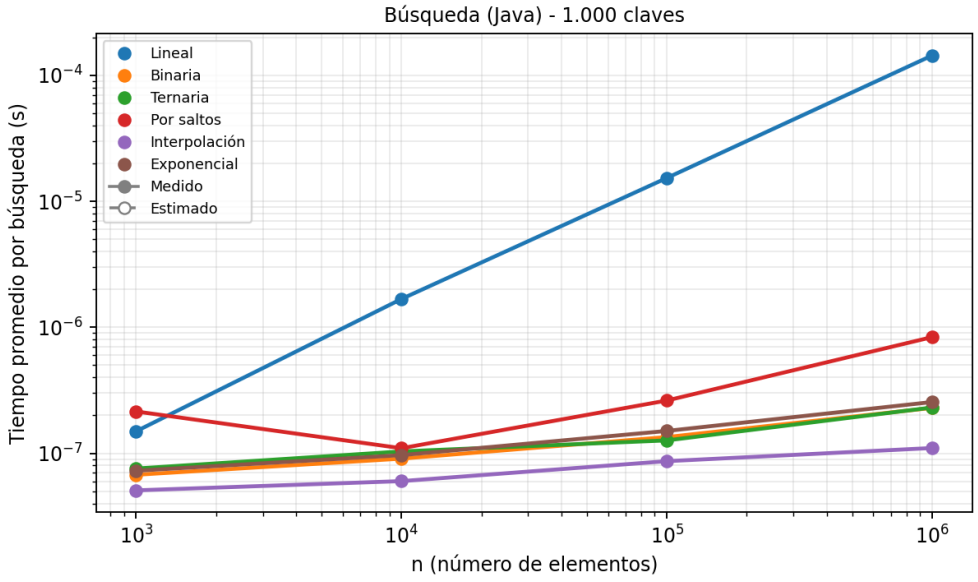
4.1. Crecimiento del tiempo con n

Análisis.

- **Búsqueda lineal ($O(n)$).** Es la única que crece como una recta de pendiente 1 en la gráfica log-log: el factor medido es ×9 a ×13 por salto frente al ×10 teórico. Por eso sirve como *referencia*: no hace ningún supuesto sobre los datos, es la cota superior contra la que se mide cuánto ahorra aprovechar el orden. Con $n = 1.000.000$ tarda 134 μs por búsqueda en Java (A) y 7,74 ms en Python (A); las demás van de 70 ns a 635 ns en Java (A).
- **Búsqueda por saltos ($O(\sqrt{n})$).** Crece ≈ ×3 por cada ×10 en n (×3,04 y ×3,19 en Java, ×3,37 y ×2,84 en Python, en el último salto), coincidiendo con el $\sqrt{10} = 3{,}16$ teórico. Es mucho más rápida que la lineal, pero queda entre entre 5 y 26 veces por detrás de las

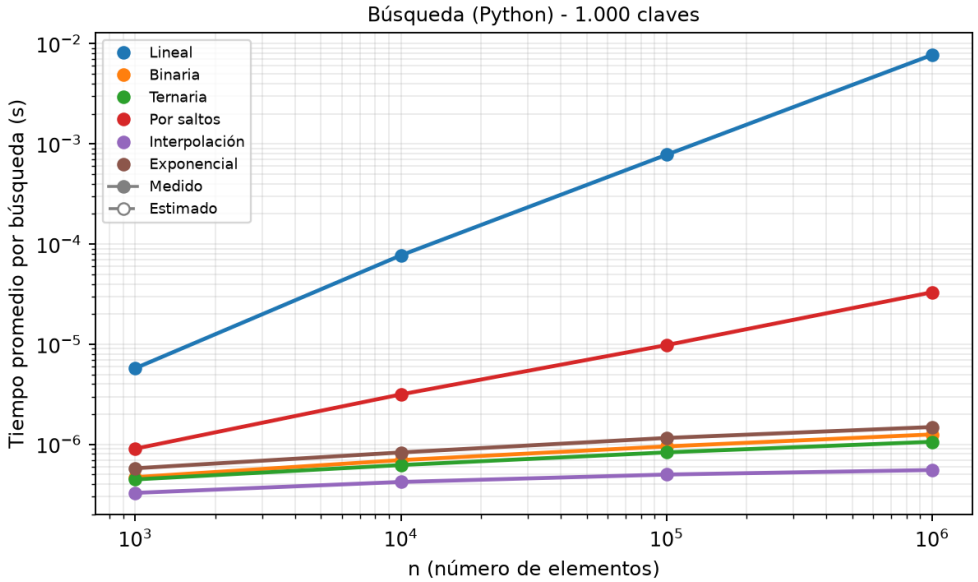


(a) Equipo A.

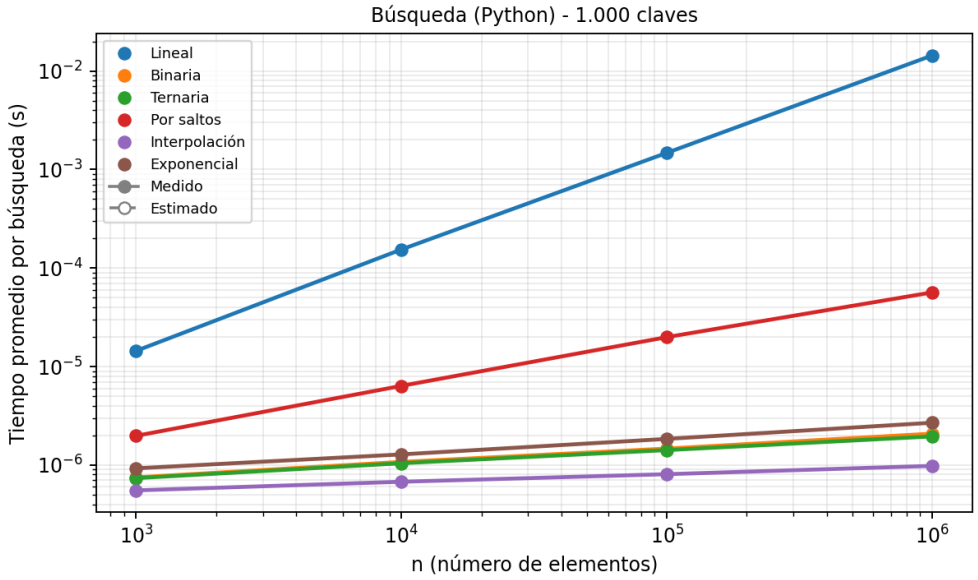


(b) Equipo B.

Figura 7: Tiempo promedio por búsqueda frente a n en Java (log-log).



(a) Equipo A.



(b) Equipo B.

Figura 8: Tiempo promedio por búsqueda frente a n en Python (log-log).

Tabla 6: Factor de crecimiento del tiempo promedio por búsqueda al multiplicar n por 10, frente al factor teórico ($\times 10$ lineal; $\times 1{,}33$, $\times 1{,}25$, $\times 1{,}2$ logarítmico; $\times 3{,}16$ para $O(\sqrt{n})$; $\times 1{,}13$, $\times 1{,}09$, $\times 1{,}06$ para $O(\log \log n)$).

Equipo A

Java

Algoritmo	$1.000 \to 10.000$	$10.000 \to 100.000$	$100.000 \to 1.000.000$
Lineal	$\times 9{,}59$ (teór. $\times 10$)	$\times 10$ (teór. $\times 10$)	$\times 12{,}1$ (teór. $\times 10$)
Binaria	$\times 2{,}17$ (teór. $\times 1{,}33$)	$\times 0{,}828$ (teór. $\times 1{,}25$)	$\times 1{,}6$ (teór. $\times 1{,}2$)
Ternaria	$\times 1{,}45$ (teór. $\times 1{,}33$)	$\times 1{,}36$ (teór. $\times 1{,}25$)	$\times 1{,}71$ (teór. $\times 1{,}2$)
Por saltos	$\times 0{,}748$ (teór. $\times 3{,}16$)	$\times 2{,}57$ (teór. $\times 3{,}16$)	$\times 3{,}04$ (teór. $\times 3{,}16$)
Interpolación	$\times 1{,}42$ (teór. $\times 1{,}13$)	$\times 1{,}02$ (teór. $\times 1{,}09$)	$\times 1{,}68$ (teór. $\times 1{,}06$)
Exponencial	$\times 1{,}12$ (teór. $\times 1{,}33$)	$\times 1{,}98$ (teór. $\times 1{,}25$)	$\times 1{,}54$ (teór. $\times 1{,}2$)

Python

Algoritmo	$1.000 \to 10.000$	$10.000 \to 100.000$	$100.000 \to 1.000.000$
Lineal	$\times 13{,}4$ (teór. $\times 10$)	$\times 10{,}1$ (teór. $\times 10$)	$\times 9{,}93$ (teór. $\times 10$)
Binaria	$\times 1{,}48$ (teór. $\times 1{,}33$)	$\times 1{,}37$ (teór. $\times 1{,}25$)	$\times 1{,}32$ (teór. $\times 1{,}2$)
Ternaria	$\times 1{,}39$ (teór. $\times 1{,}33$)	$\times 1{,}34$ (teór. $\times 1{,}25$)	$\times 1{,}28$ (teór. $\times 1{,}2$)
Por saltos	$\times 3{,}48$ (teór. $\times 3{,}16$)	$\times 3{,}11$ (teór. $\times 3{,}16$)	$\times 3{,}37$ (teór. $\times 3{,}16$)
Interpolación	$\times 1{,}29$ (teór. $\times 1{,}13$)	$\times 1{,}19$ (teór. $\times 1{,}09$)	$\times 1{,}11$ (teór. $\times 1{,}06$)
Exponencial	$\times 1{,}44$ (teór. $\times 1{,}33$)	$\times 1{,}4$ (teór. $\times 1{,}25$)	$\times 1{,}29$ (teór. $\times 1{,}2$)

Equipo B

Java

Algoritmo	$1.000 \to 10.000$	$10.000 \to 100.000$	$100.000 \to 1.000.000$
Lineal	$\times 11{,}2$ (teór. $\times 10$)	$\times 9{,}13$ (teór. $\times 10$)	$\times 9{,}39$ (teór. $\times 10$)
Binaria	$\times 1{,}34$ (teór. $\times 1{,}33$)	$\times 1{,}48$ (teór. $\times 1{,}25$)	$\times 1{,}71$ (teór. $\times 1{,}2$)
Ternaria	$\times 1{,}37$ (teór. $\times 1{,}33$)	$\times 1{,}22$ (teór. $\times 1{,}25$)	$\times 1{,}82$ (teór. $\times 1{,}2$)
Por saltos	$\times 0{,}511$ (teór. $\times 3{,}16$)	$\times 2{,}39$ (teór. $\times 3{,}16$)	$\times 3{,}19$ (teór. $\times 3{,}16$)
Interpolación	$\times 1{,}18$ (teór. $\times 1{,}13$)	$\times 1{,}44$ (teór. $\times 1{,}09$)	$\times 1{,}27$ (teór. $\times 1{,}06$)
Exponencial	$\times 1{,}33$ (teór. $\times 1{,}33$)	$\times 1{,}56$ (teór. $\times 1{,}25$)	$\times 1{,}7$ (teór. $\times 1{,}2$)

Python

Algoritmo	$1.000 \to 10.000$	$10.000 \to 100.000$	$100.000 \to 1.000.000$
Lineal	$\times 10{,}7$ (teór. $\times 10$)	$\times 9{,}56$ (teór. $\times 10$)	$\times 9{,}74$ (teór. $\times 10$)
Binaria	$\times 1{,}44$ (teór. $\times 1{,}33$)	$\times 1{,}36$ (teór. $\times 1{,}25$)	$\times 1{,}42$ (teór. $\times 1{,}2$)
Ternaria	$\times 1{,}42$ (teór. $\times 1{,}33$)	$\times 1{,}36$ (teór. $\times 1{,}25$)	$\times 1{,}38$ (teór. $\times 1{,}2$)
Por saltos	$\times 3{,}22$ (teór. $\times 3{,}16$)	$\times 3{,}12$ (teór. $\times 3{,}16$)	$\times 2{,}84$ (teór. $\times 3{,}16$)
Interpolación	$\times 1{,}22$ (teór. $\times 1{,}13$)	$\times 1{,}19$ (teór. $\times 1{,}09$)	$\times 1{,}21$ (teór. $\times 1{,}06$)
Exponencial	$\times 1{,}39$ (teór. $\times 1{,}33$)	$\times 1{,}44$ (teór. $\times 1{,}25$)	$\times 1{,}46$ (teór. $\times 1{,}2$)

logarítmicas con $n = 10^6$ (635 ns frente a 135 ns de la binaria en Java A; 33 μs frente a 1,26 μs en Python A).

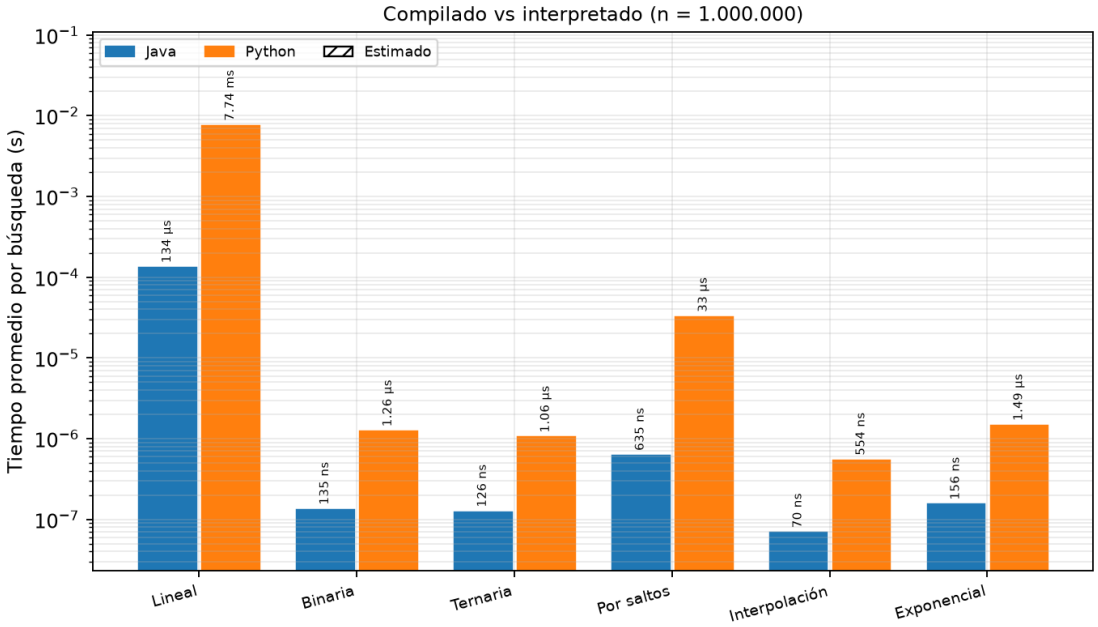
- **Binaria, ternaria y exponencial ($O(\log n)$).** Casi no crecen: por cada $\times 10$ en n el tiempo se multiplica por $\approx$ 1,2 a 1,8 (con algún valor aislado entre $\times 0{,}8$ y $\times 2{,}2$), cerca del $\times 1{,}2$ a $\times 1{,}33$ que predice $\log_2 n$ (de 10 a 20 iteraciones). Las desviaciones vienen de que son tiempos de decenas de nanosegundos, donde el ruido pesa, y de que al crecer el arreglo los últimos accesos de cada búsqueda caen más lejos en memoria. La ternaria hace $\log_3 n$ iteraciones pero con dos comparaciones cada una ($2 \log_3 n \approx 1{,}26 \log_2 n$ comparaciones), así que no gana a la binaria en comparaciones; en la práctica quedaron empatadas dentro del ruido (en Python la ternaria es algo más rápida porque ejecuta menos iteraciones del ciclo interpretado).
- **Interpolación ($O(\log \log n)$ promedio).** Fue la más rápida con $n = 10^6$ en los dos lenguajes y los dos equipos (70 ns y 110 ns en Java; 554 ns y 981 ns en Python), porque los datos son uniformes: estima la posición y casi siempre acierta en pocos intentos. Su ventaja es modesta frente a la binaria (unas 2 veces) porque $\log \log n$ y $\log n$ ya son valores pequeños con $n = 10^6$ ($\approx$ 4,3 frente a $\approx$ 20); su factor de crecimiento ($\times 1{,}0$ a $\times 1{,}7$) no se distingue del de las logarítmicas dentro del ruido de medición. Con datos no uniformes se degradaría hacia $O(n)$.
- **A partir de qué n se nota la diferencia con la lineal.** En Python ya con $n = 1.000$ la lineal es unas 12 veces más lenta que la binaria (5,76 μs frente a 0,47 μs en A); en Java, con $n = 1.000$ la diferencia es de solo 2,5 veces (116 ns frente a 47 ns), con $n = 10.000$ ya son 11 veces y con $n = 1.000.000$ son unas 1.000 veces (134 μs frente a 135 ns).

4.2. Compilado frente a interpretado

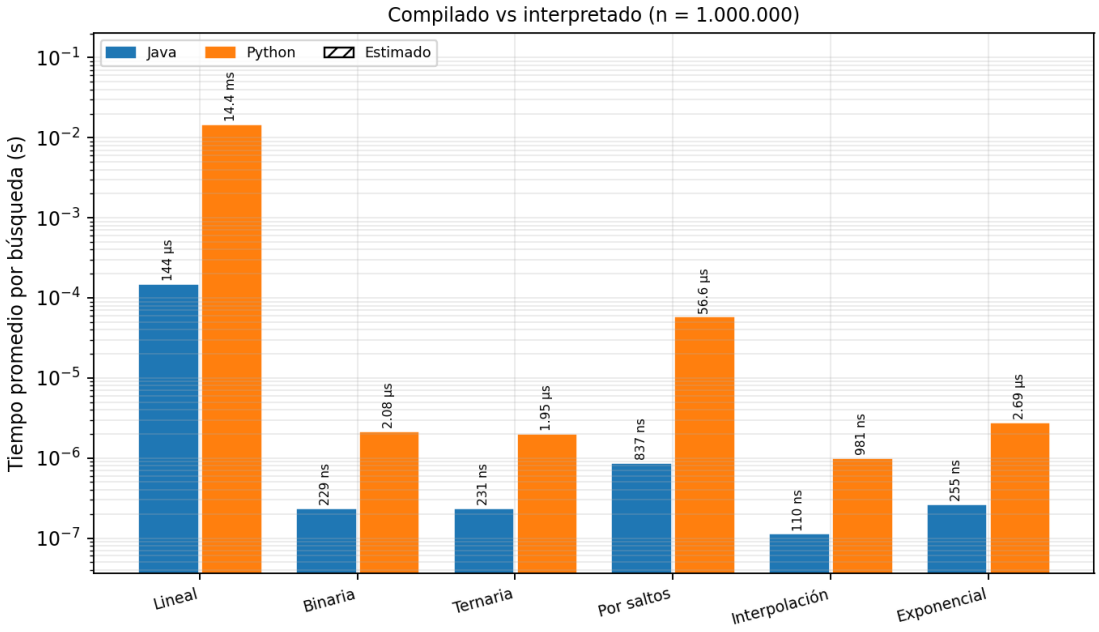
Análisis. El cociente Python/Java con $n = 1.000.000$ es de $\times 8$ a $\times 10$ para las búsquedas logarítmicas y de interpolación (binaria $\times 9{,}3$, ternaria $\times 8{,}4$, interpolación $\times 7{,}9$, exponencial $\times 9{,}5$ en el equipo A; $\times 9{,}1$, $\times 8{,}5$, $\times 8{,}9$ y $\times 10{,}5$ en el B), pero de $\times 52$ a $\times 100$ para las que recorren muchos elementos: lineal ($\times 57{,}7$ en A, $\times 99{,}9$ en B) y por saltos ($\times 52$ en A, $\times 67{,}6$ en B). La razón es la misma del ordenamiento: cuando una búsqueda solo ejecuta $\approx$ 20 iteraciones, una parte importante del tiempo en Java son esperas de memoria (cada sondeo cae lejos del anterior en un arreglo de 4 MB), un costo que Python paga igual; solo se multiplica el costo de ejecutar las instrucciones, y la brecha se comprime. Cuando hay 10^5 o 10^6 iteraciones consecutivas, la diferencia de costo *por iteración* (código de máquina frente a bytecode) se acumula casi por completo, y el ciclo lineal de Java, muy simple y sobre memoria contigua, es además fácil de optimizar para el compilador. Es una interpretación de los datos; no la medimos de forma aislada.

4.3. Posición de la clave

Las Tablas 7–10 muestran el tiempo promedio por búsqueda cuando la misma clave está al inicio (índice 0), en el centro (índice $n/2$) y al final (índice $n - 1$) del arreglo de $n = 1.000.000$, para cada equipo y lenguaje.



(a) Equipo A.



(b) Equipo B.

Figura 9: Java frente a Python en búsqueda, $n = 1.000.000$ (tiempo promedio por búsqueda).

Tabla 7: Equipo A, Java: tiempo promedio por búsqueda según la posición de la clave ($n = 1.000.000$).

Algoritmo	Inicio	Centro	Final
Lineal	0,933 ± 0,4 ns	72,6 ± 1,5 μs	156 ± 11 μs
Binaria	22,3 ± 1,1 ns	21,2 ± 1,9 ns	23,1 ± 0,59 ns
Ternaria	23,6 ± 0,55 ns	24,3 ± 1,3 ns	33,7 ± 15 ns
Por saltos	3,19 ± 0,18 ns	300 ± 3,6 ns	1,21 ± 0,058 μs
Interpolación	3,12 ± 0,33 ns	7,23 ± 0,2 ns	3,28 ± 0,28 ns
Exponencial	27,3 ± 20 ns	48,3 ± 14 ns	34,8 ± 4,2 ns

Tabla 8: Equipo A, Python: tiempo promedio por búsqueda según la posición de la clave ($n = 1.000.000$).

Algoritmo	Inicio	Centro	Final
Lineal	66,6 ± 2,5 ns	5,12 ± 0,028 ms	10,2 ± 0,061 ms
Binaria	759 ± 10 ns	793 ± 17 ns	804 ± 8,7 ns
Ternaria	696 ± 12 ns	806 ± 13 ns	770 ± 8,1 ns
Por saltos	105 ± 1,3 ns	18,9 ± 0,1 μs	65 ± 1,9 μs
Interpolación	108 ± 4,4 ns	312 ± 3,4 ns	125 ± 6,7 ns
Exponencial	36,9 ± 2,1 ns	1,03 ± 0,012 μs	1,2 ± 0,0089 μs

Tabla 9: Equipo B, Java: tiempo promedio por búsqueda según la posición de la clave ($n = 1.000.000$).

Algoritmo	Inicio	Centro	Final
Lineal	0,864 ± 0,14 ns	91 ± 3,4 μs	174 ± 5 μs
Binaria	33,3 ± 7,4 ns	30,4 ± 0,099 ns	39,8 ± 14 ns
Ternaria	36 ± 2,3 ns	44 ± 10 ns	38,4 ± 2,7 ns
Por saltos	4,89 ± 0,16 ns	443 ± 4,3 ns	1,57 ± 0,009 μs
Interpolación	5,59 ± 0,28 ns	30,3 ± 12 ns	5,79 ± 0,06 ns
Exponencial	41,9 ± 45 ns	91,5 ± 36 ns	60,6 ± 11 ns

Tabla 10: Equipo B, Python: tiempo promedio por búsqueda según la posición de la clave ($n = 1.000.000$).

Algoritmo	Inicio	Centro	Final
Lineal	95,8 ± 1,9 ns	9,23 ± 0,21 ms	18,7 ± 0,29 ms
Binaria	1,31 ± 0,029 μs	1,35 ± 0,02 μs	1,4 ± 0,0084 μs
Ternaria	1,2 ± 0,0093 μs	1,45 ± 0,0091 μs	1,4 ± 0,025 μs
Por saltos	162 ± 5,3 ns	29,5 ± 0,25 μs	104 ± 2,6 μs
Interpolación	147 ± 2,2 ns	534 ± 6 ns	186 ± 1,1 ns
Exponencial	59,7 ± 1,2 ns	1,66 ± 0,017 μs	2 ± 0,0037 μs

Análisis. El patrón es el mismo en los dos lenguajes y en los dos equipos:

- **Cambia mucho en la lineal y en la de saltos.** Ambas recorren el arreglo desde el inicio, así que su costo es proporcional a cuánto hay que avanzar. La lineal con la clave al inicio hace una sola comparación (≈ 1 ns en Java, 67 ns en Python, que es básicamente el costo de la llamada); en el centro revisa $n/2$ elementos (72,6 μs en Java A) y al final los n (156 μs), de modo que el centro cuesta casi la mitad que el final. La de saltos con la clave al inicio la encuentra en el primer bloque (3 ns); en el centro da ≈ 500 saltos de $\sqrt{n} = 1.000$ posiciones (300 ns), y al final da ≈ 1.000 saltos y luego recorre un bloque completo (1,21 μs), por lo que final/centro es ≈ 4 y no 2.
- **Casi no cambia en la binaria y la ternaria.** Ambas siempre parten el intervalo, sin importar dónde esté la clave, y necesitan $\approx \log n$ pasos en cualquier posición (22, 21 y 23 ns en Java A para la binaria). Un detalle: con $n = 10^6$, el primer punto medio de la binaria es el índice 499.999, y la clave «centro» está en el índice 500.000, así que *no* se encuentra en el primer intento y ese caso cuesta lo mismo que los otros dos; el mejor caso $O(1)$ de la binaria habría requerido ubicar la clave exactamente en el punto medio. Además, como se busca la misma clave 1.000 veces seguidas, el predictor de ramificaciones y la caché se «aprenden» el recorrido, lo que hace estos tiempos algo menores que con claves variadas.
- **Interpolación: depende poco, pero no es constante.** Con la clave al inicio o al final la fórmula de interpolación cae exactamente en el extremo (3 ns); en el centro la estimación cae cerca pero no exactamente, por lo que son necesarias algunas iteraciones más (7 ns en Java A; 312 ns frente a 108 y 125 ns en Python A).
- **Exponencial: crece con el logaritmo de la posición.** Si la clave está en el índice 0 se detecta de inmediato; en el centro o al final debe duplicar el índice $\approx \log_2 i$ veces y luego hacer una búsqueda binaria en el último intervalo, por eso el centro y el final cuestan casi lo mismo entre sí (1,03 y 1,20 μs en Python A). En Java, el valor «inicio» (27 ns en A, 42 ns en B) es mucho mayor que el ≈ 1 ns que se esperaría de una salida inmediata; una hipótesis es que en el calentamiento el JIT nunca vio ejecutarse esa salida temprana y, la primera vez, tuvo que *desoptimizar* y recompilar el método, costo que al promediar sobre 1.000 búsquedas se nota. No lo comprobamos.

5. Punto 3: análisis comparativo

5.1. Comparativos frente a no comparativos

Límite inferior teórico. Cualquier algoritmo de ordenamiento que solo obtiene información del orden relativo comparando pares de elementos puede modelarse como un árbol de decisión binario: cada hoja es una de las $n!$ permutaciones posibles de la entrada, y cada comparación es un nodo interno con dos resultados. Un árbol binario con $n!$ hojas tiene altura al menos $\log_2(n!)$, y por la aproximación de Stirling $\log_2(n!) = n\log_2 n - n\log_2 e + O(\log n) = \Theta(n \log n)$. Por tanto, **ningún ordenamiento por comparaciones puede hacerlo en menos de $\Omega(n \log n)$ comparaciones en el peor caso** [1]. Con $n = 10^6$ eso significa al menos $\approx 1{,}85 \times 10^7$ comparaciones.

Por qué Radix Sort y Bucket Sort no están sujetos a él. El argumento anterior supone que la única operación que distingue elementos es la comparación. Radix Sort y Bucket Sort *nunca comparan dos elementos entre sí*: Radix extrae un dígito y lo usa como índice de un arreglo de conteo; Bucket calcula la cubeta de un valor con una fórmula aritmética sobre el propio valor. Es información adicional sobre los datos (que son enteros de rango acotado o de distribución conocida) y, a cambio, ordenan en tiempo $O(d(n+b))$ y $O(n+k)$ respectivamente, es decir, lineal en n.

Tabla 11: Tiempos de ordenamiento con $n = 1.000.000$ (entrada aleatoria). Promedio ± desviación estándar.

Equipo A

Algoritmo	Tipo	Java	Python
Insertion Sort	Comparativo	36,7 ± 0,07 s	∼1,91 h (est.)
Dual-Pivot QuickSort	Comparativo	51,7 ± 0,35 ms	623 ± 4,2 ms
Merge Sort	Comparativo	77,7 ± 5,8 ms	986 ± 14 ms
Heap Sort	Comparativo	86 ± 1,1 ms	1,97 ± 0,11 s
Radix Sort	No comparativo	18,2 ± 0,57 ms	1,34 ± 0,097 s
Bucket Sort	No comparativo	9,08 ± 2,8 ms	486 ± 50 ms

Equipo B

Algoritmo	Tipo	Java	Python
Insertion Sort	Comparativo	55,3 ± 23 s	∼3,81 h (est.)
Dual-Pivot QuickSort	Comparativo	76,9 ± 2,4 ms	1,05 ± 0,048 s
Merge Sort	Comparativo	124 ± 6,1 ms	1,75 ± 0,036 s
Heap Sort	Comparativo	129 ± 4,1 ms	3,06 ± 0,11 s
Radix Sort	No comparativo	50,2 ± 1,4 ms	1,89 ± 0,37 s
Bucket Sort	No comparativo	32,5 ± 12 ms	833 ± 74 ms

Comparación de tiempos. En Java, Bucket Sort y Radix Sort fueron los dos más rápidos en ambos equipos: Bucket Sort tardó 9,08 ms (A) y 32,5 ms (B), unas 5,7 y 2,4 veces menos que el mejor de los comparativos (Dual-Pivot QuickSort: 51,7 ms y 76,9 ms); Radix Sort, 18,2 ms y 50,2 ms (2,8 y 1,5 veces menos). Merge Sort y Heap Sort, los más lentos entre los $O(n \log n)$, tardaron entre 78 ms y 129 ms. En Python la ventaja se reduce: Bucket Sort sigue siendo el mejor (0,486 s en A y 0,833 s en B), pero solo 1,3 veces más rápido que Dual-Pivot QuickSort, y Radix Sort (1,34 s y 1,89 s) es *más lento* que Dual-Pivot QuickSort y Merge Sort (aunque más rápido que Heap Sort). La explicación es que Radix Sort hace $d = 8$ pasadas completas con tres ciclos cada una, y en un lenguaje interpretado el número de operaciones elementales pesa más que la ventaja asintótica (8 pasadas frente a $\log_2 10^6 \approx 20$ niveles): la ventaja teórica de ser lineal solo se materializa si las operaciones son baratas, como en Java.

Supuesto de cada algoritmo y qué pasaría si no se cumple.

- **Radix Sort** supone claves enteras (o representables como cadenas de dígitos) con una cantidad *fija y pequeña* d de dígitos; su costo es $O(d(n + b))$. En este taller $d = 8$. Con números de 16 dígitos se necesitarían 16 pasadas, es decir, el tiempo *se duplicaría* en el mejor de los casos: a partir de nuestras mediciones, unos 36 ms en lugar de 18 ms en Java (A) y 1,34 s →∼2,7 s en Python (A), **estimaciones**, no mediciones. Además, 16 dígitos no caben en un `int` de Java (hay que usar `long`). Si d creciera hasta $\sim \log n$ (por ejemplo, enteros de 64 bits), Radix Sort perdería su ventaja sobre los comparativos.
- **Bucket Sort** supone que los valores están distribuidos *aproximadamente de manera uniforme* en el intervalo [mín, máx], de modo que cada cubeta recibe ≈ 1 elemento. Si los valores se concentran (por ejemplo, casi todos cerca del mínimo y unos pocos valores extremos), la mayoría cae en una o pocas cubetas, y el Insertion Sort interno trabaja sobre casi todo el arreglo: el costo pasa de $O(n)$ a $O(n^2)$. Con nuestras mediciones se puede estimar la magnitud: Insertion Sort sobre 10^6 elementos aleatorios tarda 36,7 s en Java (A), frente a 9,08 ms del Bucket Sort con datos uniformes, es decir, alrededor de **4.000 veces más** en el peor caso (estimación).

5.2. ¿Ordenar o no ordenar?

Se tiene un arreglo *desordenado* de 1.000.000 de elementos sobre el que se deben hacer k búsquedas. Hay dos estrategias: (A) k búsquedas lineales, con costo $k \cdot t_{\text{lineal}}$; (B) ordenar una vez con el mejor algoritmo medido y luego hacer k búsquedas binarias, con costo $T_{\text{orden}} + k \cdot t_{\text{binaria}}$. Ordenar conviene cuando

$$T_{\text{orden}} + k\, t_{\text{binaria}} \;<\; k\, t_{\text{lineal}} \quad\Longleftrightarrow\quad k \;>\; k^* = \frac{T_{\text{orden}}}{t_{\text{lineal}} - t_{\text{binaria}}}. \tag{2}$$

Como T_{orden} se toma de ordenar un arreglo aleatorio de 10^6 elementos, t_{lineal} y t_{binaria} son los tiempos promedio por búsqueda medidos con $n = 1.000.000$ (Punto 2). La búsqueda lineal no aprovecha el orden, por lo que su costo por búsqueda es el mismo en el arreglo desordenado que en el ordenado. El mejor algoritmo de ordenamiento medido fue, en los dos lenguajes y los dos equipos, **Bucket Sort**.

Tabla 12: Cálculo del umbral k^* con $n = 1.000.000$.

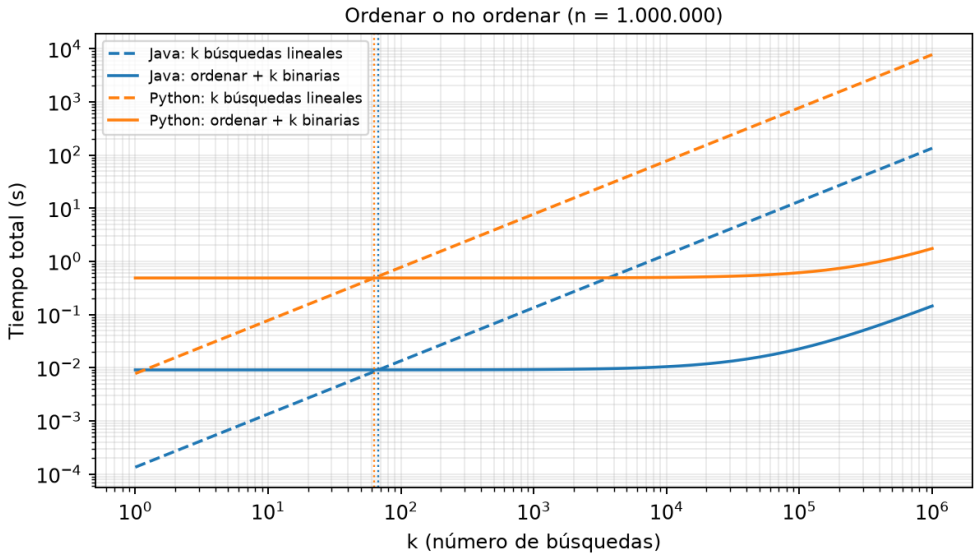
Equipo	Lenguaje	Mejor ordenamiento	T_{orden}	t_{lineal}	t_{binaria}	k^*
A	Java	Bucket Sort	9,08 ms	134 μs	135 ns	67,71 $\Rightarrow$ 68
A	Python	Bucket Sort	486 ms	7,74 ms	1,26 μs	62,83 $\Rightarrow$ 63
B	Java	Bucket Sort	32,5 ms	144 μs	229 ns	226,1 $\Rightarrow$ 227
B	Python	Bucket Sort	833 ms	14,4 ms	2,08 μs	57,85 $\Rightarrow$ 58

Cálculo (ecuación (2)).

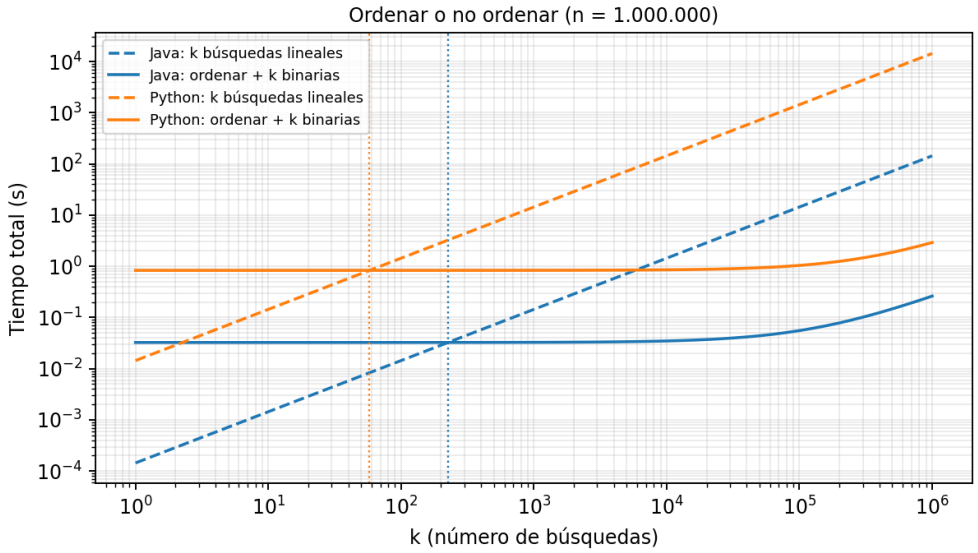
$$\begin{aligned}
\text{Java, A:}\quad & k^* = \frac{9{,}08 \text{ ms}}{134{,}2\ \mu\text{s} - 0{,}135\ \mu\text{s}} = \frac{9{,}08 \times 10^{-3}}{1{,}341 \times 10^{-4}} \approx 67{,}7 \;\Rightarrow\; k \geq 68,\\
\text{Python, A:}\quad & k^* = \frac{0{,}486 \text{ s}}{7{,}74 \text{ ms} - 1{,}26\ \mu\text{s}} \approx 62{,}8 \;\Rightarrow\; k \geq 63,\\
\text{Java, B:}\quad & k^* = \frac{32{,}5 \text{ ms}}{144{,}0\ \mu\text{s} - 0{,}229\ \mu\text{s}} \approx 226{,}1 \;\Rightarrow\; k \geq 227,\\
\text{Python, B:}\quad & k^* = \frac{0{,}833 \text{ s}}{14{,}4 \text{ ms} - 2{,}08\ \mu\text{s}} \approx 57{,}9 \;\Rightarrow\; k \geq 58.
\end{aligned}$$

Interpretación. k^* responde cuántas búsquedas hacen falta para que el costo único de ordenar se «pague». Con menos de k^* búsquedas (por ejemplo $k = 1$, cuando ordenar costaría 9 ms frente a 134 μs de buscar linealmente en Java A), conviene la búsqueda lineal; con más, conviene ordenar. A partir de k^* la diferencia crece rápido: con $k = 10^6$, la estrategia A tarda ≈ 134 s en Java (A) y la B $\approx 0{,}14$ s. Los umbrales son del mismo orden de magnitud (decenas a pocos cientos) y por una razón simple: ordenar con Bucket Sort equivale, en tiempo, a unas 60–230 búsquedas lineales.

- **Por qué difiere entre lenguajes y equipos.** En Python, tanto ordenar como buscar linealmente son ciclos interpretados con costos similares por elemento, y el cociente $T_{\text{orden}}/t_{\text{lineal}}$ es estable (63 y 58 en los dos equipos). En Java, la búsqueda lineal es un ciclo simple sobre memoria contigua (muy rápido, $\approx 0{,}2$ ns por elemento revisado) mientras que Bucket Sort hace escrituras dispersas; en el equipo A ordenar equivale a 68 búsquedas lineales y en el B a 227, porque Bucket Sort empeoró mucho más al pasar de A a B ($\times 3{,}6$ en tiempo) que la búsqueda lineal ($\times 1{,}07$): una pasada secuencial no depende tanto de la jerarquía de memoria como una escritura dispersa.
- **Salvedades.** (i) Los valores de k^* usan el tiempo medio por búsqueda del conjunto de 1.000 claves, mitad presentes y mitad ausentes; si las claves reales estuvieran casi siempre



(a) Equipo A.



(b) Equipo B.

Figura 10: Costo total de cada estrategia según k ($n = 1.000.000$). Las líneas verticales punteadas marcan k^*.

al inicio, la lineal sería mucho más barata y k^* mucho mayor. (ii) Si después de ordenar se necesitarán los datos en su orden original, habría que conservar una copia. (iii) Si los datos cambian entre búsquedas, hay que volver a ordenar y la ecuación (2) deja de aplicar.

6. Comparación entre los dos equipos

La Tabla 13 muestra el cociente de tiempos equipo B / equipo A con $n = 1.000.000$ (valores mayores que $\times 1$ indican que B fue más lento).

Tabla 13: Cociente de tiempos B/A con $n = 1.000.000$ (entrada aleatoria para ordenamiento; 1.000 claves para búsqueda).

Algoritmo	Java	Python
Ordenamiento (aleatoria)		
Insertion Sort	×1,51	∼×1,99
Dual-Pivot QuickSort	×1,49	×1,69
Merge Sort	×1,59	×1,77
Heap Sort	×1,5	×1,56
Radix Sort	×2,76	×1,41
Bucket Sort	×3,58	×1,71
Búsqueda (1.000 claves)		
Lineal	×1,07	×1,86
Binaria	×1,69	×1,66
Ternaria	×1,83	×1,84
Por saltos	×1,32	×1,71
Interpolación	×1,57	×1,77
Exponencial	×1,64	×1,81

- **Mismas conclusiones, distinta escala.** En todos los casos el equipo B fue más lento (entre ×1,1 y ×3,6; la mayoría entre ×1,3 y ×2,0), pero el orden de los algoritmos, los factores de crecimiento, los efectos del orden de entrada, la brecha Java–Python y los umbrales k^* de Python son cualitativamente los mismos. Las conclusiones, por tanto, no dependen del equipo.
- **Donde más se separan los equipos.** Los algoritmos cuyo costo está dominado por accesos dispersos a memoria (Bucket Sort ×3,6 y Radix Sort ×2,8 en Java) se degradan más en B que los dominados por cómputo (Merge, Heap, Dual-Pivot: ×1,5–1,6), y la búsqueda lineal en Java (acceso secuencial) casi no cambia (×1,07). Es consistente con que B tenga una jerarquía de memoria menos favorable que A, aunque sin perfilar el hardware esa es solo una hipótesis.
- **Variabilidad.** Algunos tiempos de Java con $n = 1.000$ (por ejemplo, Heap Sort en A) se alejan de lo esperado por efectos de calentamiento del JIT; con $n \geq 10.000$ las desviaciones estándar son pequeñas frente al promedio en la mayoría de los casos (ver Anexo).

7. Uso de inteligencia artificial

Se utilizó **Claude Code** (Anthropic) como herramienta de apoyo en las siguientes partes del trabajo:

- **Implementación de los algoritmos y de los programas de medición:** primera versión del código en Java y Python, adaptada de los artículos de GeeksforGeeks de la Tabla 1, y

de los programas de medición (calentamiento, repeticiones, verificación y estimación de tiempos).

- **Generación de gráficas y tablas:** scripts en Python (matplotlib) que leen los CSV y producen las gráficas; script que genera las tablas en LaTeX de este documento a partir de los CSV de los dos equipos.
- **Investigación y redacción:** consulta de las complejidades teóricas y redacción de un borrador de los análisis y del documento LaTeX, a partir de los resultados medidos.

Las mediciones se ejecutaron en los dos equipos por el grupo; la herramienta no generó ni modificó ningún dato experimental. **[El grupo debe revisar este texto y ajustarlo para que describa con exactitud el uso real que hizo, y revisar con sus propias palabras los análisis antes de entregar.]**

8. Conclusiones

1. **El comportamiento coincide con la teoría.** Al multiplicar n por 10, Insertion Sort multiplicó su tiempo por ≈ 100 ($O(n^2)$), Merge Sort, Heap Sort y Dual-Pivot QuickSort por ≈ 11–13 ($O(n \log n)$) y Radix Sort por ≈ 10 ($O(n)$), en Java y en Python y en ambos equipos. Las desviaciones (hasta $\times 17$–19 en Radix y Bucket Sort con $n = 10^6$) se explican por la jerarquía de memoria y por los costos del JIT con n pequeño, no por un error de complejidad.
2. **El orden de entrada puede cambiar la complejidad.** Insertion Sort pasó de 361 ms a 53 μs ($n = 10^5$, Java, A) con entrada ascendente, pero fue el doble de lento con descendente. Dual-Pivot QuickSort con pivotes en los extremos degeneró a $O(n^2)$ con datos ordenados y falló por desbordamiento de pila (Java, $n = 10^5$) o por el límite de recursión (Python, $n = 10^4$). Merge, Heap, Radix y Bucket Sort no cambiaron de complejidad con el orden.
3. **Mejores algoritmos.** Con $n = 10^6$ aleatorio, Bucket Sort fue el más rápido (9,08 ms en Java A; 0,486 s en Python A), seguido de Radix Sort en Java y de Dual-Pivot QuickSort en Python. Entre los comparativos, Dual-Pivot QuickSort ganó con datos aleatorios, pero es el único que no es seguro con datos ordenados. Los no comparativos superan al límite $\Omega(n \log n)$ porque no comparan, a cambio de supuestos sobre los datos (dígitos fijos, distribución uniforme).
4. **Compilado frente a interpretado.** Java fue entre $\times 8$ y $\times 250$ más rápido. La brecha es pequeña ($\times 8$–14) en búsquedas logarítmicas y en Merge Sort/Dual-Pivot QuickSort, y grande ($\times 50$–250) en lo que repite muchas iteraciones simples (búsqueda lineal y por saltos, Insertion, Radix y Bucket Sort).
5. **Búsqueda.** Con datos uniformes, la interpolación fue la más rápida; binaria, ternaria y exponencial quedaron empatadas dentro del ruido; la de saltos creció como $\sqrt{n}$ y la lineal, como n. La posición de la clave solo importa para los métodos que recorren desde el inicio (lineal, saltos y, en menor medida, exponencial e interpolación).
6. **¿Ordenar o no ordenar?** Para $n = 10^6$ conviene ordenar con Bucket Sort y usar búsqueda binaria a partir de $k^* \approx 58$–227 búsquedas, según lenguaje y equipo.
7. **Limitaciones.** Cinco repeticiones por medición; los valores estimados (Insertion Sort en Python con $n = 10^6$ y Dual-Pivot QuickSort con entrada ordenada) escalan con un modelo teórico y deben leerse como aproximaciones; las explicaciones sobre caché, JIT y predicción de ramificaciones son hipótesis razonables que no verificamos con herramientas de perfilado.

Referencias

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3.ª ed., MIT Press, 2009. Cap. 8, Teorema 8.1 (cota inferior de ordenamiento por comparaciones).

[2] GeeksforGeeks, artículos de algoritmos de ordenamiento y búsqueda (ver Tabla 1). `https://www.geeksforgeeks.org/`

A. Tablas completas de resultados

Promedio ± desviación estándar muestral de 5 repeticiones. «∼ . . . (est.)» indica un valor estimado, no medido.

A.1. Equipo A

Tabla 14: Equipo A, ordenamiento en Java.

Algoritmo	Entrada	$n = 1.000$	$n = 10.000$	$n = 100.000$	$n = 1.000.000$
Insertion Sort	Aleatoria	47,1 ± 7 µs	3,5 ± 0,062 ms	361 ± 0,75 ms	36,7 ± 0,07 s
	Ascendente	3,18 ± 2,7 µs	14,1 ± 11 µs	53,1 ± 0,59 µs	572 ± 77 µs
	Descendente	91,9 ± 2,8 µs	7,47 ± 0,12 ms	729 ± 7,3 ms	1,23 ± 0,00037 min
Dual-Pivot QuickSort	Aleatoria	44,6 ± 13 µs	417 ± 70 µs	4,67 ± 0,26 ms	51,7 ± 0,35 ms
	Ascendente	138 ± 3,8 µs	14,6 ± 0,12 ms	∼ 1,46 s (est.)	∼ 2,43 min (est.)
	Descendente	135 ± 1,6 µs	15,1 ± 0,44 ms	∼ 1,51 s (est.)	∼ 2,52 min (est.)
Merge Sort	Aleatoria	46,8 ± 7,6 µs	537 ± 19 µs	6,47 ± 0,17 ms	77,7 ± 5,8 ms
	Ascendente	20,9 ± 1,6 µs	174 ± 4,3 µs	1,9 ± 0,049 ms	28,8 ± 16 ms
	Descendente	21,1 ± 2,5 µs	176 ± 3,1 µs	1,89 ± 0,037 ms	21 ± 0,19 ms
Heap Sort	Aleatoria	210 ± 364,17 µs	572 ± 40 µs	6,81 ± 0,12 ms	86 ± 1,1 ms
	Ascendente	34,9 ± 2,1 µs	372 ± 8,5 µs	4,01 ± 0,13 ms	46,2 ± 3,2 ms
	Descendente	32,2 ± 2,9 µs	387 ± 9 µs	4,19 ± 0,079 ms	45,4 ± 0,16 ms
Radix Sort	Aleatoria	26,2 ± 4,4 µs	200 ± 5,4 µs	1,93 ± 0,1 ms	18,2 ± 0,57 ms
	Ascendente	22,9 ± 2,3 µs	188 ± 6,6 µs	2 ± 0,17 ms	18,4 ± 0,18 ms
	Descendente	22,8 ± 1,5 µs	201 ± 10 µs	2,49 ± 0,42 ms	17,9 ± 0,22 ms
Bucket Sort	Aleatoria	26,3 ± 11 µs	167 ± 64 µs	1,96 ± 0,88 ms	9,08 ± 2,8 ms
	Ascendente	10,8 ± 6,2 µs	52,6 ± 5,1 µs	729 ± 303,30 µs	5,8 ± 0,91 ms
	Descendente	8,97 ± 1,4 µs	58,8 ± 6,9 µs	577 ± 26 µs	6,04 ± 0,12 ms

Tabla 15: Equipo A, ordenamiento en Python.

Algoritmo	Entrada	$n = 1.000$	$n = 10.000$	$n = 100.000$	$n = 1.000.000$
Insertion Sort	Aleatoria	4,95 ± 0,033 ms	611 ± 12 ms	1,15 ± 0,076 min	∼ 1,91 h (est.)
	Ascendente	30,9 ± 1 µs	300 ± 4,1 µs	3,22 ± 0,21 ms	30 ± 0,16 ms
	Descendente	9,82 ± 0,17 ms	1,16 ± 0,013 s	2,07 ± 0,058 min	∼ 3,45 h (est.)
Dual-Pivot QuickSort	Aleatoria	263 ± 18 µs	3,91 ± 0,051 ms	47,5 ± 0,15 ms	623 ± 4,2 ms
	Ascendente	4,48 ± 0,032 ms	∼ 448 ms (est.)	∼ 44,8 s (est.)	∼ 1,24 h (est.)
	Descendente	4,58 ± 0,032 ms	∼ 458 ms (est.)	∼ 45,8 s (est.)	∼ 1,27 h (est.)
Merge Sort	Aleatoria	472 ± 16 µs	6,36 ± 0,038 ms	77,6 ± 0,27 ms	986 ± 14 ms
	Ascendente	367 ± 3 µs	4,97 ± 0,022 ms	58,8 ± 0,068 ms	678 ± 6,1 ms
	Descendente	360 ± 3 µs	4,87 ± 0,019 ms	57,9 ± 0,18 ms	665 ± 2,1 ms
Heap Sort	Aleatoria	658 ± 15 µs	9,81 ± 0,15 ms	126 ± 0,62 ms	1,97 ± 0,11 s
	Ascendente	670 ± 10 µs	9,45 ± 0,082 ms	121 ± 0,91 ms	1,46 ± 0,012 s
	Descendente	589 ± 6,8 µs	8,52 ± 0,03 ms	111 ± 0,45 ms	1,41 ± 0,029 s
Radix Sort	Aleatoria	655 ± 102,37 µs	7,23 ± 0,49 ms	69,6 ± 0,51 ms	1,34 ± 0,097 s
	Ascendente	635 ± 28 µs	7 ± 0,1 ms	68,3 ± 0,96 ms	924 ± 30 ms
	Descendente	611 ± 4,7 µs	6,88 ± 0,064 ms	68,3 ± 1,1 ms	1,07 ± 0,13 s
Bucket Sort	Aleatoria	186 ± 13 µs	2,38 ± 0,063 ms	26,3 ± 1,2 ms	486 ± 50 ms
	Ascendente	174 ± 8,2 µs	2,17 ± 0,18 ms	21,3 ± 0,24 ms	224 ± 6,6 ms
	Descendente	183 ± 2,1 µs	2,18 ± 0,023 ms	22,2 ± 0,2 ms	262 ± 36 ms

Tabla 16: Equipo A, búsqueda en Java (tiempo por búsqueda, 1.000 claves).

Algoritmo	$n = 1.000$	$n = 10.000$	$n = 100.000$	$n = 1.000.000$
Lineal	116 ± 3,3 ns	1,11 ± 0,028 μs	11,1 ± 0,32 μs	134 ± 20 μs
Binaria	47,1 ± 6,1 ns	102 ± 90 ns	84,7 ± 13 ns	135 ± 50 ns
Ternaria	37,3 ± 12 ns	54,3 ± 11 ns	74 ± 18 ns	126 ± 54 ns
Por saltos	109 ± 59 ns	81,4 ± 4,6 ns	209 ± 11 ns	635 ± 46 ns
Interpolación	28,9 ± 2,2 ns	41,1 ± 9,6 ns	41,8 ± 15 ns	70 ± 42 ns
Exponencial	45,6 ± 7,7 ns	51 ± 8,2 ns	101 ± 14 ns	156 ± 52 ns

Tabla 17: Equipo A, búsqueda en Python (tiempo por búsqueda, 1.000 claves).

Algoritmo	$n = 1.000$	$n = 10.000$	$n = 100.000$	$n = 1.000.000$
Lineal	5,76 ± 0,23 μs	77,2 ± 2,6 μs	779 ± 8,3 μs	7,74 ± 0,099 ms
Binaria	471 ± 22 ns	696 ± 52 ns	954 ± 198,41 ns	1,26 ± 0,39 μs
Ternaria	446 ± 17 ns	621 ± 24 ns	832 ± 93 ns	1,06 ± 0,26 μs
Por saltos	905 ± 12 ns	3,15 ± 0,017 μs	9,8 ± 0,072 μs	33 ± 0,74 μs
Interpolación	328 ± 9,8 ns	421 ± 18 ns	500 ± 96 ns	554 ± 131,21 ns
Exponencial	577 ± 13 ns	831 ± 23 ns	1,16 ± 0,13 μs	1,49 ± 0,28 μs

Tabla 18: Equipo A: registro de valores estimados y fallos.

Lenguaje	Algoritmo	Entrada	n	Estado	Nota
Java	DualPivotQuickSort	asc	100.000	ERROR_StackOverflowError	StackOverflowError at n=100000; scaled from n=10000
Java	DualPivotQuickSort	asc	1.000.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=10000
Java	DualPivotQuickSort	desc	100.000	ERROR_StackOverflowError	StackOverflowError at n=100000; scaled from n=10000
Java	DualPivotQuickSort	desc	1.000.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=10000
Python	DualPivotQuickSort	asc	10.000	ERROR_RecursionError	RecursionError at n=10000 (limit 1000); scaled from n=1000
Python	DualPivotQuickSort	asc	100.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=1000
Python	DualPivotQuickSort	asc	1.000.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=1000
Python	DualPivotQuickSort	desc	10.000	ERROR_RecursionError	RecursionError at n=10000 (limit 1000); scaled from n=1000
Python	DualPivotQuickSort	desc	100.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=1000
Python	DualPivotQuickSort	desc	1.000.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=1000
Python	InsertionSort	desc	1.000.000	ESTIMATED	predicted 12413s > limit 300s; scaled from n=100000
Python	InsertionSort	random	1.000.000	ESTIMATED	predicted 6890s > limit 300s; scaled from n=100000

A.2. Equipo B

Tabla 19: Equipo B, ordenamiento en Java.

Algoritmo	Entrada	$n = 1.000$	$n = 10.000$	$n = 100.000$	$n = 1.000.000$
Insertion Sort	Aleatoria	49,4 ± 2,4 µs	3,47 ± 0,24 ms	372 ± 1,9 ms	55,3 ± 23 s
	Ascendente	3,4 ± 0,78 µs	8,67 ± 0,26 µs	76,6 ± 10 µs	813 ± 84 µs
	Descendente	87,1 ± 7,3 µs	7,19 ± 0,094 ms	792 ± 18 ms	1,55 ± 0,02 min
Dual-Pivot QuickSort	Aleatoria	51,1 ± 4,8 µs	598 ± 73 µs	6,61 ± 0,023 ms	76,9 ± 2,4 ms
	Ascendente	178 ± 8,4 µs	16,4 ± 0,054 ms	∼1,64 s (est.)	∼2,73 min (est.)
	Descendente	177 ± 0,89 µs	17,4 ± 0,22 ms	∼1,74 s (est.)	∼2,9 min (est.)
Merge Sort	Aleatoria	75 ± 4,3 µs	810 ± 25 µs	9,94 ± 0,16 ms	124 ± 6,1 ms
	Ascendente	39,9 ± 17 µs	277 ± 14 µs	3,2 ± 0,037 ms	42,2 ± 2,3 ms
	Descendente	38,2 ± 1,5 µs	321 ± 6,1 µs	3,8 ± 0,058 ms	48,3 ± 2,4 ms
Heap Sort	Aleatoria	84,5 ± 31 µs	928 ± 24 µs	10,6 ± 0,065 ms	129 ± 4,1 ms
	Ascendente	41,5 ± 3,3 µs	597 ± 91 µs	5,65 ± 0,038 ms	62,3 ± 1,2 ms
	Descendente	49,4 ± 3,4 µs	535 ± 6,2 µs	5,81 ± 0,01 ms	64,1 ± 1,2 ms
Radix Sort	Aleatoria	74,2 ± 31 µs	481 ± 26 µs	4,59 ± 0,046 ms	50,2 ± 1,4 ms
	Ascendente	49,8 ± 2 µs	451 ± 4,7 µs	4,59 ± 0,04 ms	50,5 ± 1,7 ms
	Descendente	48 ± 1,4 µs	448 ± 20 µs	4,51 ± 0,035 ms	48,3 ± 0,7 ms
Bucket Sort	Aleatoria	45,8 ± 6,3 µs	224 ± 47 µs	1,87 ± 0,17 ms	32,5 ± 12 ms
	Ascendente	14,7 ± 2,6 µs	111 ± 20 µs	1,01 ± 0,071 ms	12,5 ± 0,61 ms
	Descendente	12,9 ± 2,4 µs	102 ± 3,1 µs	1,03 ± 0,02 ms	13,9 ± 1,5 ms

Tabla 20: Equipo B, ordenamiento en Python.

Algoritmo	Entrada	$n = 1.000$	$n = 10.000$	$n = 100.000$	$n = 1.000.000$
Insertion Sort	Aleatoria	9,74 ± 0,1 ms	1,01 ± 0,013 s	2,29 ± 0,45 min	∼3,81 h (est.)
	Ascendente	51,9 ± 8,4 µs	512 ± 7,1 µs	5,4 ± 0,57 ms	53,4 ± 0,3 ms
	Descendente	18,1 ± 0,062 ms	1,98 ± 0,049 s	3,69 ± 0,39 min	∼6,15 h (est.)
Dual-Pivot QuickSort	Aleatoria	434 ± 11 µs	6,19 ± 0,036 ms	77,3 ± 1,2 ms	1,05 ± 0,048 s
	Ascendente	9,11 ± 0,36 ms	∼911 ms (est.)	∼1,52 min (est.)	∼2,53 h (est.)
	Descendente	8,39 ± 0,074 ms	∼839 ms (est.)	∼1,4 min (est.)	∼2,33 h (est.)
Merge Sort	Aleatoria	802 ± 22 µs	10,4 ± 0,11 ms	130 ± 2,6 ms	1,75 ± 0,036 s
	Ascendente	718 ± 35 µs	9,21 ± 0,13 ms	110 ± 2,5 ms	1,24 ± 0,044 s
	Descendente	687 ± 23 µs	8,9 ± 0,12 ms	107 ± 3,4 ms	1,2 ± 0,03 s
Heap Sort	Aleatoria	1,17 ± 0,024 ms	16,9 ± 0,22 ms	223 ± 4,9 ms	3,06 ± 0,11 s
	Ascendente	1,08 ± 0,014 ms	15,6 ± 0,27 ms	206 ± 2,5 ms	2,48 ± 0,017 s
	Descendente	915 ± 28 µs	14,4 ± 0,24 ms	188 ± 2,3 ms	2,34 ± 0,014 s
Radix Sort	Aleatoria	792 ± 52 µs	9,89 ± 0,65 ms	109 ± 2,3 ms	1,89 ± 0,37 s
	Ascendente	1,49 ± 0,11 ms	15 ± 0,26 ms	166 ± 8,2 ms	1,86 ± 0,16 s
	Descendente	1,36 ± 0,023 ms	13,8 ± 0,13 ms	151 ± 7,4 ms	1,7 ± 0,11 s
Bucket Sort	Aleatoria	450 ± 27 µs	4,59 ± 0,014 ms	57 ± 1,3 ms	833 ± 74 ms
	Ascendente	578 ± 404,55 µs	4,17 ± 0,019 ms	44,4 ± 0,27 ms	473 ± 16 ms
	Descendente	411 ± 35 µs	4,29 ± 0,079 ms	45,7 ± 0,45 ms	481 ± 12 ms

Tabla 21: Equipo B, búsqueda en Java (tiempo por búsqueda, 1.000 claves).

Algoritmo	$n = 1.000$	$n = 10.000$	$n = 100.000$	$n = 1.000.000$
Lineal	150 ± 3,7 ns	1,68 ± 0,38 µs	15,3 ± 1,6 µs	144 ± 7,1 µs
Binaria	67,6 ± 2,5 ns	90,7 ± 1,6 ns	135 ± 13 ns	229 ± 43 ns
Ternaria	75,7 ± 11 ns	104 ± 12 ns	127 ± 11 ns	231 ± 58 ns
Por saltos	215 ± 126,90 ns	110 ± 6 ns	262 ± 28 ns	837 ± 34 ns
Interpolación	50,8 ± 3,7 ns	60,1 ± 4,3 ns	86,6 ± 11 ns	110 ± 24 ns
Exponencial	72,7 ± 1,3 ns	96,7 ± 2,1 ns	151 ± 29 ns	255 ± 36 ns

Tabla 22: Equipo B, búsqueda en Python (tiempo por búsqueda, 1.000 claves).

Algoritmo	$n = 1.000$	$n = 10.000$	$n = 100.000$	$n = 1.000.000$
Lineal	14,4 ± 0,035 μs	155 ± 3,3 μs	1,48 ± 0,073 ms	14,4 ± 0,45 ms
Binaria	749 ± 15 ns	1,08 ± 0,026 μs	1,47 ± 0,17 μs	2,08 ± 0,32 μs
Ternaria	736 ± 17 ns	1,04 ± 0,018 μs	1,42 ± 0,14 μs	1,95 ± 0,31 μs
Por saltos	1,98 ± 0,21 μs	6,37 ± 0,2 μs	19,9 ± 0,46 μs	56,6 ± 2,5 μs
Interpolación	553 ± 7,9 ns	677 ± 29 ns	808 ± 144,14 ns	981 ± 197,13 ns
Exponencial	926 ± 26 ns	1,28 ± 0,021 μs	1,84 ± 0,22 μs	2,69 ± 0,34 μs

Tabla 23: Equipo B: registro de valores estimados y fallos.

Lenguaje	Algoritmo	Entrada	n	Estado	Nota
Java	DualPivotQuickSort	asc	100.000	ERROR_StackOverflowError	StackOverflowError at n=100000; scaled from n=10000
Java	DualPivotQuickSort	asc	1.000.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=10000
Java	DualPivotQuickSort	desc	100.000	ERROR_StackOverflowError	StackOverflowError at n=100000; scaled from n=10000
Java	DualPivotQuickSort	desc	1.000.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=10000
Python	DualPivotQuickSort	asc	10.000	ERROR_RecursionError	RecursionError at n=10000 (limit 1000); scaled from n=1000
Python	DualPivotQuickSort	asc	100.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=1000
Python	DualPivotQuickSort	asc	1.000.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=1000
Python	DualPivotQuickSort	desc	10.000	ERROR_RecursionError	RecursionError at n=10000 (limit 1000); scaled from n=1000
Python	DualPivotQuickSort	desc	100.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=1000
Python	DualPivotQuickSort	desc	1.000.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=1000
Python	InsertionSort	desc	1.000.000	ESTIMATED	predicted 22131s > limit 300s; scaled from n=100000
Python	InsertionSort	random	1.000.000	ESTIMATED	predicted 13718s > limit 300s; scaled from n=100000